



UNIVERSIDAD DE LOS LAGOS

PROYECTO SISTEMA PARA UNIVERSIDAD

PROYECTO MULTIASIGNATURA

DEPARTAMENTO DE CIENCIAS DE LA INGENIERÍA
INGENIERÍA CIVIL EN INFORMÁTICA
ASIGNATURA
CAMPUS OSORNO, CHILE

Autor
Autor

23 de septiembre de 2026



5 **UNIVERSIDAD ACREDITADA**
Septiembre de 2021 - Septiembre de 2026
Gestión Institucional - Docencia de Pregrado
Investigación - Vinculación con el Medio
AVANZADA

www.ulagos.cl

Índice general

1. Formulación del Proyecto	1
1.1. Introducción	1
1.2. Objetivos del Proyecto	1
1.2.1. Objetivo General	2
1.2.2. Objetivos Específicos	2
1.3. Metodología de Trabajo	2
1.3.1. Objetivo Específico 1: Fase de Concepción y Formulación	2
1.3.2. Objetivo Específico 2: Fase de Diseño y Modelado	3
1.3.3. Objetivo Específico 3: Fase de Implementación del Prototipo	3
1.3.4. Objetivo Específico 4: Fase de Validación y Pruebas (Simulación)	4
1.4. Equipo de Trabajo	4
1.4.1. Responsabilidades Compartidas por Todo el Equipo	6
1.5. Planificación Gantt del Proyecto	7
1.6. Justificación y Aporte	9
1.6.1. Justificación del Proyecto	9
1.6.2. Utilidad del Sistema para la Universidad	10
1.6.3. Aporte del Sistema a la Universidad	10
1.7. Análisis de Viabilidad del Proyecto	12
1.7.1. Viabilidad Técnica	12
1.7.2. Viabilidad Económica	13
1.7.3. Viabilidad Operativa	14
1.7.4. Viabilidad de Plazos (Cronograma – 15 Meses)	14
1.8. Metodología de Desarrollo	16
1.9. Planificación Temporal	18
1.10. Diagramas UML	22
1.10.1. Diagrama de Casos de Uso (Interacción de Usuarios)	22
1.10.2. Diagrama de Actividades (Asignación a Proyecto)	24
1.10.3. Diagrama de Estados (Ciclo de Vida de un Proyecto)	25
2. Modelo de Datos	26
2.1. Requisitos	26
2.2. Modelo Entidad Relación	27

2.2.1.	Definición de Entidades	27
2.2.2.	Definición de Relaciones	28
2.2.3.	Modelo Entidad Resultante	32
2.2.4.	Modelo Entidad Relación sin Herencia	33
2.3.	Modelo Relacional	34
2.3.1.	Conversión Relaciones N-arias	34
2.3.2.	Conversión Relaciones N:N	35
2.3.3.	Conversión Relaciones 1:N	36
2.3.4.	Conversión Relaciones 1:1	38
2.3.5.	Conversión Relaciones Reflexivas y otras	39
2.3.6.	Modelo Relacional Resultante	40
2.4.	Normalización	41
2.4.1.	Tablas que Cumplen con la Tercera Forma Normal (3NF)	41
2.4.2.	Tablas que NO se Encuentran en Tercera Forma Normal (3NF)	42
2.4.3.	Modelo Relacional Normalizado	45
3.	Lenguaje de Definición de Datos	46
3.1.	Creación de Tablas	46
3.2.	Insert SQL	48
4.	Lenguaje de Manipulación de Datos	54
4.1.	Restricciones	54
4.2.	Consultas y Vistas	56

Índice de figuras

1.1. Desglose semanal de la Carta Gantt del proyecto, visibilizando objetivos, actividades y tareas. Se evidencia el estado actual de progreso y el paralelismo operativo aplicado.	8
1.2. Caso de Uso interacción Profesor y Alumno	22
1.3. Diagrama de actividad de incorporar un alumno de postgrado a un proyecto	24
1.4. Estado de la entidad proyectos	25
2.1. Herencia Persona	27
2.2. Relación Supervisar	29
2.3. Relación Dirigir	29
2.4. Relación Laborar	29
2.5. Relación Trabajar	30
2.6. Relación Cursar	30
2.7. Relación Pertenece	30
2.8. Relación Administrar	31
2.9. Relación Asesorar	31
2.10. Modelo Entidad Relación de la universidad	32
2.11. MER de la universidad aplicado Eliminación del supertipo a la herencia	33
2.12. Relación Supervisar	34
2.13. Relación Dirigir	35
2.14. Relación Laborar	35
2.15. Relación Trabajar	36
2.16. Relación Cursar	37
2.17. Relación Pertenece	37
2.18. Relación Administrar	38
2.19. Relación Asesorar	39
2.20. Modelo Relacional de la universidad	40
2.21. Normalización Tabla Departamento	44
2.22. Modelo Relacional Normalizado de la universidad	45

Índice de tablas

1.1. Matriz de precedencias estructurada según nomenclatura WBS, evidenciando dependencias y tareas concurrentes.	9
1.2. Planificación Temporal del Proyecto Orientado a Prototipo	19
1.3. Especificación del Caso de Uso para la asignación controlada de alumnos y supervisores en investigaciones.	23
2.1. Entidad Proyecto	28
2.2. Entidad Postgrado	28
2.3. Entidad Departamento	28
2.4. MR de la Relación Supervisar	34
2.5. MR de la Relación Dirigir	35
2.6. MR de la Relación Laborar	36
2.7. MR de la Relación Trabajar	36
2.8. MR de la Relación Cursar	37
2.9. MR de la Relación Pertenece	38
2.10. MR de la Relación Administrar	38
2.11. MR de la Relación Asesorar	39

Índice de algoritmos

3.1. Eliminación de las Tablas	46
3.2. Creaciones de Tablas	46
3.3. Datos para la tabla Profesor	48
3.4. Datos para la tabla Postgrado	49
3.5. Datos para la tabla Proyecto	49
3.6. Datos para la tabla Departamento	50
3.7. Datos para la tabla Alumno	50
3.8. Datos para la tabla Dirigir	51
3.9. Datos para la tabla Laborar	51
3.10. Datos para la tabla Trabajar	52
3.11. Datos para la tabla Supervisar	52
3.12. Datos insertados en la tabla Supervisar	53
4.1. Restricción Fecha Inicio Proyecto	54
4.2. Insert Impedido por la restricción chk_proyecto_fechainicio_valida	54
4.3. Eliminar Restricción Fecha Inicio Proyecto	54
4.4. Restricción Presupuesto no negativo	55
4.5. Coherencia de Fechas del Proyecto	55
4.6. Fecha de Nacimiento Válida	55
4.7. Rango del Profesor	55
4.8. Auto-Asesoría Inválida	55
4.9. Porcentaje de Dedicación Válido	55
4.10. Número de departamento positivo	56
4.11. Número de despacho positivo	56
4.12. Código positivo	56
4.13. Profesores de máximo rango	56
4.14. Resultado Código 4.13	56
4.15. Proyectos sin fecha de fin	56
4.16. Resultado Código 4.15	57
4.17. Directores de Departamento (Join Implícito)	57
4.18. Proyectos y sus Directores (INNER JOIN)	57
4.19. Presupuesto total por patrocinador	57
4.20. Profesores que dirigen más de un proyecto	58
4.21. Alumnos en programas de Magíster (Uso de IN)	58

4.22. Departamentos con profesores que trabajan al 100 % (Uso de EXISTS)	58
4.23. Alumnos sin proyecto (Usando LEFT JOIN)	59
4.24. Alumnos sin proyecto (Usando NOT EXISTS)	59
4.25. Creación de una vista ProyectoDetallado	59
4.26. Uso de la vista para filtrar proyectos	60
4.27. Supervisión de Alumnos en Proyectos	60
4.28. Postgrado con más alumnos	60

Capítulo 1

Formulación del Proyecto

1.1. Introducción

El presente documento se ha elaborado en respuesta a la solicitud de [Nombre del Cliente o Empresa Cliente] para el desarrollo de un sistema de base de datos web destinado a gestionar la información académica y de investigación de su institución. Comprendemos la necesidad de contar con una plataforma robusta, escalable y eficiente que permita administrar de manera integrada los datos relativos a profesores, alumnos, proyectos, departamentos y asignaturas.

Este informe tiene como objetivo principal presentar una propuesta detallada del modelo de datos que servirá como pilar fundamental para la construcción del sistema. Un modelo de datos bien definido es crucial para garantizar la integridad de la información, facilitar el desarrollo de funcionalidades complejas, optimizar las consultas y asegurar la futura adaptabilidad del sistema a nuevas necesidades.

La información contenida en las siguientes secciones ha sido cuidadosamente analizada a partir de los requisitos proporcionados. Se expondrá la identificación de las entidades principales, sus atributos y las relaciones que existen entre ellas. Esta estructura ha sido pensada para ser comprensible tanto por el equipo de gerencia, para la toma de decisiones estratégicas, como por el equipo de desarrollo, quienes se encargarán de la implementación técnica.

Confiamos en que este análisis y la propuesta de diseño que se presenta a continuación constituirán una base sólida para el exitoso desarrollo del sistema de base de datos web que su institución requiere. Estamos a su disposición para discutir cualquier aspecto de este informe y para colaborar estrechamente en las siguientes fases del proyecto.

Perfecto. A continuación, te presento una propuesta para el objetivo general y los objetivos específicos del proyecto, considerando las etapas que mencionaste:

1.2. Objetivos del Proyecto

La arquitectura metodológica de este trabajo asume los lineamientos de la iniciativa CDIO, marco que postula un ciclo de vida estructurado para el desarrollo de sistemas ingenieriles[cite:

1]. Considerando los límites operativos del encargo, el proyecto transita exclusivamente por las fases de Concepción, Diseño e Implementación (CDI)[cite: 1], delegando la etapa de Operación comercial y sustituyéndola por una validación funcional a nivel de prototipo computacional.

1.2.1. Objetivo General

Desarrollar un sistema de base de datos web para la gestión de información académica e investigativa en el contexto de [Nombre del Cliente o Empresa Cliente], con el propósito de optimizar el acceso concurrente y centralizar la administración de los registros institucionales.

1.2.2. Objetivos Específicos

1. **Concebir:** Formular los requerimientos técnicos y operacionales mediante el diagnóstico de las necesidades organizacionales del cliente, delimitando la factibilidad del sistema de información.
2. **Diseñar:** Modelar la arquitectura de software y los esquemas lógicos de la base de datos relacional que estructurarán la solución tecnológica subyacente.
3. **Implementar:** Construir el artefacto informático a nivel de prototipo, integrando los lenguajes de programación web/móvil y los motores de datos seleccionados en un entorno de desarrollo controlado.
4. Evaluar la consistencia y rendimiento del prototipo funcional mediante simulaciones de carga y pruebas estructurales, verificando las métricas de calidad sin contemplar su despliegue en un entorno operativo de producción.

1.3. Metodología de Trabajo

El desglose de actividades se estructura en subordinación a los propósitos planteados, estableciendo un flujo lógico desde la abstracción teórica hasta la verificación empírica del código fuente. Esta configuración asegura una trazabilidad coherente a lo largo del ciclo de vida del software, alineando el desarrollo del prototipo con los requerimientos académicos.

1.3.1. Objetivo Específico 1: Fase de Concepción y Formulación

Actividades y Tareas

1. Diagnóstico de Requerimientos y Extracción de Entidades:

- 1.1 Levantar los requerimientos funcionales extrayendo las entidades principales del dominio del problema (Profesor, Proyecto, Alumno, ProgramaPostgrado, Departamento, Asignatura).
- 1.2 Evaluar la factibilidad tecnológica preliminar para la plataforma web, delimitando el alcance del prototipo.
2. Consolidación Documental y Planificación Temporal:
 - 2.1 Estructurar el informe técnico inicial, argumentando la justificación del desarrollo y definiendo la matriz de planificación temporal (Carta Gantt) para las iteraciones.
 - 2.2 Revisar y validar la comprensión de los requerimientos, asegurando que el modelo mental concuerda con las necesidades institucionales.

1.3.2. Objetivo Específico 2: Fase de Diseño y Modelado

Actividades y Tareas

1. Modelado Conceptual y Lógico de Datos:
 - 1.1 Proyectar el Modelo Entidad-Relación (MER), definiendo atributos clave (por ejemplo, el RUN como identificador único) y estableciendo las cardinalidades y restricciones (1:N, M:N).
 - 1.2 Transformar el MER al modelo relacional lógico, ejecutando un proceso de normalización riguroso hasta la Tercera Forma Normal (3FN) para evitar anomalías transaccionales y dependencias transitivas.
2. Arquitectura de Interfaces y Experiencia de Usuario:
 - 2.1 Elaborar wireframes de bajo nivel para esquematizar la estructura y navegación del sistema.
 - 2.2 Construir mockups de alta fidelidad que proyecten la apariencia visual, considerando el diseño diferenciado de interfaces para usuarios con privilegios limitados frente a los administradores.

1.3.3. Objetivo Específico 3: Fase de Implementación del Prototipo

Actividades y Tareas

1. Construcción y Poblamiento del Motor de Datos:
 - 1.1 Escribir los scripts DDL (Data Definition Language) para la creación de esquemas, implementando sentencias CREATE TABLE junto con sus restricciones de integridad (PRIMARY KEY, FOREIGN KEY, NOT NULL).

- 1.2 Poblar las estructuras relacionales con un conjunto de datos de prueba (mock data) representativo mediante sentencias `INSERT INTO`, preparando el terreno para las pruebas funcionales.

2. Programación de la Capa de Aplicación y Presentación:

- 2.1 Desarrollar el frontend utilizando semántica HTML5 y reglas de presentación CSS3, garantizando una correcta adaptabilidad en la visualización.
- 2.2 Codificar la lógica de negocio en el backend mediante PHP, estableciendo conectores seguros (por ejemplo, PDO o conectores nativos para PostgreSQL/MySQL) hacia el SGBD.
- 2.3 Implementar las rutinas transaccionales (CRUD) para todas las entidades y desarrollar funciones lógicas o *triggers* en la base de datos que automaticen la integridad referencial profunda.

1.3.4. Objetivo Específico 4: Fase de Validación y Pruebas (Simulación)

Actividades y Tareas

1. Ejecución de Pruebas Unitarias y de Integración:

- 1.1 Diseñar casos de prueba aislados para validar las funciones críticas del backend PHP y el comportamiento de los *triggers* en la base de datos.
- 1.2 Comprobar la correcta intercomunicación de datos (integración) entre los formularios web, el servidor de aplicaciones y las tablas relacionales.

2. Validación Estructural y de Seguridad del Prototipo:

- 2.1 Evaluar el rendimiento del motor de base de datos ejecutando consultas predefinidas y verificando el impacto en los tiempos de respuesta tras la creación de índices.
- 2.2 Ejecutar pruebas de seguridad perimetral sobre el prototipo, orientadas a mitigar vulnerabilidades comunes como la Inyección SQL (SQLi) y el Cross-Site Scripting (XSS), consolidando la validación del sistema sin requerir un paso a producción.

1.4. Equipo de Desarrollo del Proyecto y Asignación de Responsabilidades

La estructura del equipo asume una distribución funcional basada en las fases CDI, delegando responsabilidades específicas para la conceptualización y construcción del prototipo tecnológico[cite: 1]. Esta configuración sugiere un trabajo interdisciplinario donde cada rol nutre las distintas etapas de modelado e implementación.

1. Javier Rojas

- **Rol Principal:** Jefe de Proyecto & Desarrollador Full-Stack Senior.

- **Responsabilidades Clave:**

- Coordinar los *Sprints* y facilitar las ceremonias del marco metodológico Scrum.
- Supervisar la arquitectura del software, garantizando la viabilidad técnica entre el diseño y la implementación.
- Construir componentes complejos de la lógica de negocio en el backend (Objetivo 3).
- Guiar la transición entre las fases de implementación y validación, asegurando la consistencia del prototipo (Objetivo 4).

2. Carolina Silva

- **Rol Principal:** Analista de Sistemas & Diseñadora de Bases de Datos.

- **Responsabilidades Clave:**

- Dirigir la Fase de Concepción, diagnosticando requerimientos y extrayendo las entidades del dominio (Objetivo 1).
- Liderar el Modelado Conceptual y Lógico, proyectando el Modelo Entidad-Relación (MER) y ejecutando la normalización estructural hasta la 3FN (Objetivo 2).
- Elaborar la documentación técnica vinculada a la viabilidad y planificación inicial.

3. Martín Pizarro

- **Rol Principal:** Desarrollador Backend & Administrador de Base de Datos (DBA) Junior.

- **Responsabilidades Clave:**

- Materializar la estructura de datos mediante la escritura de scripts DDL y el poblamiento inicial con *mock data* (Objetivo 3).
- Codificar las rutinas transaccionales (CRUD) en PHP y establecer los conectores persistentes hacia el SGBD (Objetivo 3).
- Implementar *triggers* en el motor de base de datos para automatizar la integridad referencial.

4. Sofía Castro

- **Rol Principal:** Desarrolladora Full-Stack & Diseñadora UI/UX.

- **Responsabilidades Clave:**

- Esquematizar la experiencia de usuario mediante wireframes y mockups diferenciados por perfiles de acceso (Objetivo 2).
- Programar la capa de presentación utilizando semántica HTML5 y reglas de diseño CSS3 (Objetivo 3).
- Acoplar la interfaz gráfica con las rutinas del backend, evaluando la fluidez de interacción en el prototipo funcional.

5. Diego Valdés

- **Rol Principal:** Desarrollador Full-Stack & Soporte de Entornos.
- **Responsabilidades Clave:**
 - Configurar el entorno de desarrollo controlado, habilitando el servidor web y los intérpretes necesarios para el sistema.
 - Brindar apoyo transversal en la implementación de módulos CRUD y la escritura de funciones complementarias (Objetivo 3).
 - Ejecutar evaluaciones estructurales preliminares sobre el motor de base de datos (Objetivo 4).

6. Valentina Herrera

- **Rol Principal:** Encargada de Calidad (QA) & Documentación Técnica.
- **Responsabilidades Clave:**
 - Estructurar y ejecutar la Fase de Validación, diseñando casos de prueba unitaria y de integración para el prototipo (Objetivo 4).
 - Realizar auditorías de seguridad perimetral (mitigación de SQLi y XSS) en un entorno simulado, sin pasar a producción.
 - Consolidar la documentación técnica del proyecto, verificando la consistencia semántica y el apego a las normas académicas vigentes.

1.4.1. Responsabilidades Compartidas por Todo el Equipo

- **Comprensión de Requisitos:** Todos los miembros participarán en la discusión y comprensión de los requisitos del cliente.
- **Revisiones de Código:** Se realizarán revisiones cruzadas de código para mejorar la calidad y compartir conocimiento.
- **Pruebas:** Aunque Valentina lidera QA, todos los desarrolladores son responsables de las pruebas unitarias de su código y colaborarán en la identificación y corrección de errores.

- **Reuniones de Equipo:** Participación activa en reuniones periódicas de seguimiento, planificación y resolución de problemas, siguiendo la metodología Scrum.
- **Gestión del Conocimiento:** Compartir aprendizajes y soluciones encontradas durante el desarrollo.
- **Adaptabilidad:** Estar dispuestos a asumir tareas fuera de su rol principal si el proyecto lo requiere, dada la dinámica de un equipo pequeño y un proyecto formativo.

1.5. Planificación Gantt del Proyecto

La distribución cronológica presentada en la Figura 1.1 establece una secuencia trazable desde la abstracción de requerimientos hasta la validación empírica del prototipo computacional. El margen de tiempo semanal permite gestionar con precisión la concurrencia de tareas; por ejemplo, la estructuración del Modelo Entidad-Relación (MER) avanza en paralelo con la elaboración de la documentación técnica. Esta arquitectura de planificación podría facilitar una evaluación continua del rendimiento del equipo, mitigando el riesgo de desviaciones durante la fase de codificación.

Los meses de enero y febrero de 2026 han sido excluidos deliberadamente del flujo continuo, considerándose periodos de inactividad académica.

En la Tabla 1.1 muestra las precedencias de cada tarea de esta carta Gantt.

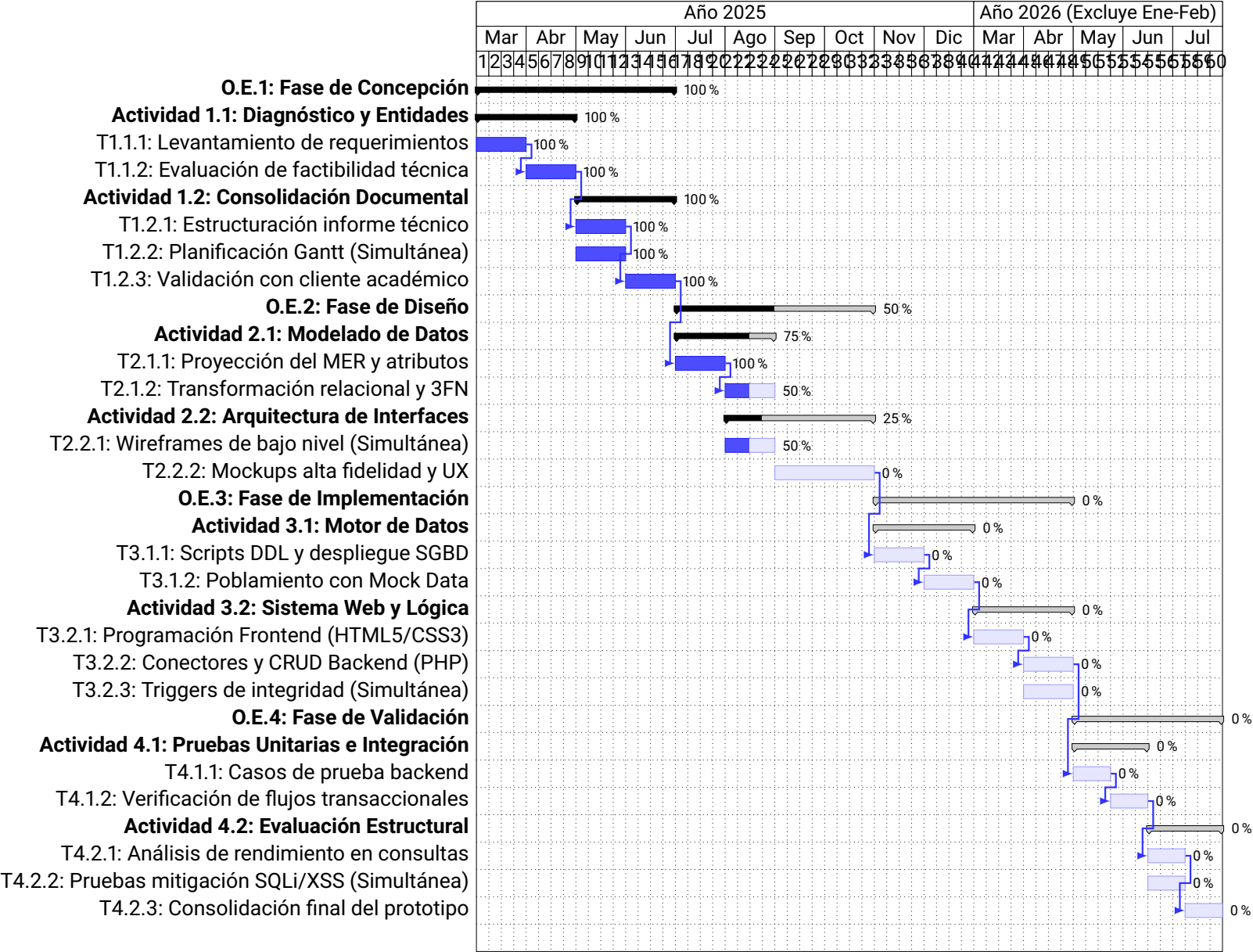


Figura 1.1: Desglose semanal de la Carta Gantt del proyecto, visibilizando objetivos, actividades y tareas. Se evidencia el estado actual de progreso y el paralelismo operativo aplicado.

WBS	Duración	Después de	Simultanea a	Antes de
T1.1.1	4	-	-	T1.1.2
T1.1.2	4	T1.1.1	-	T1.2.1, T1.2.2
T1.2.1	4	T1.1.2	T1.2.2	T1.2.3
T1.2.2	4	T1.1.2	T1.2.1	T1.2.3
T1.2.3	4	T1.2.1, T1.2.2	-	T2.1.1
T2.1.1	4	T1.2.3	-	T2.1.2, T2.2.1
T2.1.2	4	T2.1.1	T2.2.1	T2.2.2
T2.2.1	4	T2.1.1	T2.1.2	T2.2.2
T2.2.2	8	T2.1.2, T2.2.1	-	T3.1.1
T3.1.1	4	T2.2.2	-	T3.1.2
T3.1.2	4	T3.1.1	-	T3.2.1
T3.2.1	4	T3.1.2	-	T3.2.2, T3.2.3
T3.2.2	4	T3.2.1	T3.2.3	T4.1.1
T3.2.3	4	T3.2.1	T3.2.2	T4.1.1
T4.1.1	3	T3.2.2, T3.2.3	-	T4.1.2
T4.1.2	3	T4.1.1	-	T4.2.1, T4.2.2
T4.2.1	3	T4.1.2	T4.2.2	T4.2.3
T4.2.2	3	T4.1.2	T4.2.1	T4.2.3
T4.2.3	3	T4.2.1, T4.2.2	-	-

Tabla 1.1: Matriz de precedencias estructurada según nomenclatura WBS, evidenciando dependencias y tareas concurrentes.

1.6. Justificación del Proyecto, Utilidad y Aporte para la Universidad

1.6.1. Justificación del Proyecto

La gestión eficiente de la información académica y de investigación es un pilar fundamental para el funcionamiento y la excelencia de cualquier institución de educación superior. En el contexto actual, donde las universidades manejan un volumen creciente y complejo de datos relacionados con su personal docente, alumnado, proyectos de investigación, programas de postgrado y estructura departamental, la necesidad de contar con herramientas tecnológicas robustas y centralizadas se vuelve imperativa.

Actualmente, es común que esta información se encuentre dispersa en múltiples sistemas aislados, hojas de cálculo o incluso en formatos manuales, lo que genera ineficiencias, dificulta la toma de decisiones informadas, incrementa el riesgo de inconsistencias y limita la capacidad de análisis integral del quehacer universitario. El presente proyecto se justifica por la necesidad

de superar estas limitaciones mediante el desarrollo de un sistema de base de datos web diseñado a medida, que permita una administración integrada, ágil y precisa de todos estos datos críticos.

1.6.2. Utilidad del Sistema para la Universidad

El sistema de base de datos web propuesto ofrecerá una utilidad transversal a diversas áreas y niveles de la Universidad:

1. **Centralización e Integración de la Información:** Se consolidarán los datos de profesores, alumnos, proyectos, departamentos, programas de postgrado y asignaturas en una única plataforma. Esto eliminará redundancias, mejorará la consistencia de los datos y proporcionará una visión unificada de las actividades académicas y de investigación.
2. **Optimización de Procesos Administrativos:** Se simplificarán y agilizarán numerosas tareas administrativas, tales como:
 - La asignación de profesores a proyectos y la gestión de sus roles (investigador principal, investigador).
 - El seguimiento de la participación de alumnos de postgrado en proyectos como ayudantes de investigación y la asignación de sus supervisores.
 - La administración de la carga docente y la afiliación de profesores a departamentos con sus respectivos porcentajes de dedicación.
 - La gestión de la información de los programas de postgrado y la adscripción de los alumnos. La asignación de directores de departamento y asesores de alumnos.
3. **Facilitación del Acceso a la Información:** Permitirá a los usuarios autorizados (desde personal administrativo hasta directivos y los propios académicos y alumnos, según sus perfiles) acceder de manera rápida y segura a la información que necesitan, mejorando la transparencia y la capacidad de respuesta.
4. **Soporte para la Generación de Informes y Estadísticas:** Al contar con datos estructurados y centralizados, el sistema facilitará la generación de informes detallados y estadísticas relevantes para la evaluación del desempeño, la planificación estratégica y la acreditación institucional. Por ejemplo, se podrá conocer fácilmente la productividad investigadora, la carga académica por departamento o la tasa de participación de alumnos en proyectos.

1.6.3. Aporte del Sistema a la Universidad

La implementación de este sistema representará un aporte significativo para la Universidad en múltiples dimensiones:

1. **Mejora de la Eficiencia Operativa:**

- Reducción del tiempo dedicado a la búsqueda y consolidación manual de información.
- Automatización de procesos clave, disminuyendo la carga de trabajo administrativo y la probabilidad de errores humanos.
- Optimización en la asignación de recursos humanos y financieros en proyectos y departamentos.

2. Fortalecimiento de la Toma de Decisiones Estratégicas:

- Proporcionará a la gerencia y a los responsables de unidades académicas información fiable y oportuna para la planificación a corto, mediano y largo plazo.
- Permitirá identificar fortalezas, debilidades, oportunidades y amenazas en el ámbito académico y de investigación con base en evidencia concreta.
- Facilitará la evaluación del impacto de programas y políticas institucionales.

3. Impulso a la Gestión de la Investigación:

- Mejorará el seguimiento de los proyectos de investigación: sus participantes, financiamiento, plazos y supervisión.
- Facilitará la identificación de sinergias y posibles colaboraciones entre investigadores y departamentos.
- Contribuirá a la visibilidad de la producción científica de la institución.

4. Mejora en la Gestión y Apoyo Académico:

- Optimizará la administración de los programas de postgrado y el seguimiento de sus estudiantes.
- Facilitará la labor de supervisión y asesoría a los alumnos.
- Permitirá una gestión más clara de las responsabilidades docentes y de investigación del cuerpo académico.

5. Incremento de la Transparencia y Rendición de Cuentas:

- Proporcionará un registro claro y auditable de las actividades y asignaciones.
- Facilitará la preparación de informes para organismos de acreditación, entidades financiadoras y la comunidad universitaria en general.

6. Modernización Tecnológica y Base para el Futuro:

- Representa un paso adelante en la modernización de la infraestructura tecnológica de la Universidad.

- Un sistema bien diseñado y escalable podrá adaptarse a futuras necesidades e integrarse con otras plataformas institucionales, consolidando el ecosistema digital de la Universidad.

En resumen, este sistema no solo resolverá problemas operativos existentes, sino que también se convertirá en una herramienta estratégica que potenciará la capacidad de gestión, la calidad de la investigación y la formación académica, y la eficiencia general de la Universidad, contribuyendo directamente al cumplimiento de su misión y visión institucional.

1.7. Análisis de Viabilidad del Proyecto

La implementación del sistema propuesto para la Universidad se considera viable tras analizar los siguientes aspectos:

1.7.1. Viabilidad Técnica

El proyecto es técnicamente viable por las siguientes razones:

- **Tecnologías Maduras y Disponibles:** Las tecnologías propuestas para el desarrollo (HTML, CSS, JavaScript para el frontend; PHP para el backend; un Sistema Gestor de Base de Datos SQL como MySQL o PostgreSQL) son estándares en la industria, maduras, ampliamente documentadas y con una gran comunidad de soporte. Existen abundantes herramientas de desarrollo, muchas de ellas de código abierto, que facilitan su implementación.
- **Complejidad Manejable:** Si bien el sistema integra diversas entidades y relaciones, la lógica de negocio (operaciones CRUD, gestión de roles, reglas de integridad) es abordable con las tecnologías mencionadas. El modelado relacional y la normalización (hasta 3FN) asegurarán una base de datos robusta y eficiente. La implementación de funciones y triggers (PL/SQL o similar, según el SGBD) está dentro de las capacidades estándar de los SGBD modernos.
- **Recursos Humanos Calificados:** Se asume la disponibilidad de un equipo de desarrollo con experiencia en análisis de sistemas, diseño de bases de datos, desarrollo web full-stack (PHP, HTML, CSS) y gestión de proyectos.
- **Infraestructura Requerida:** La infraestructura necesaria (servidores web y de base de datos) puede ser implementada en las instalaciones de la Universidad o mediante servicios de cloud computing, ofreciendo flexibilidad y escalabilidad. Se considera que la Universidad cuenta con la capacidad para alojar o contratar estos servicios.
- **Escalabilidad y Rendimiento:** El diseño propuesto, incluyendo una correcta normalización, indexación y la posibilidad de optimizar consultas, permitirá que el sistema maneje el volumen de datos y la carga de usuarios esperada en una institución universitaria. El sistema podrá escalar horizontal o verticalmente según las necesidades futuras.

1.7.2. Viabilidad Económica

Se estima que el proyecto es económicamente viable, considerando:

Costos de Desarrollo

- **Personal:** Principal componente de costo, incluyendo analistas, diseñadores, desarrolladores (frontend y backend), especialistas en bases de datos y testers durante los 15 meses.
- **Software:** Potencialmente, costos de licencias para el SGBD o herramientas de desarrollo especializadas, aunque se priorizará el uso de software de código abierto o con licenciamiento favorable para instituciones educativas.
- **Hardware/Infraestructura:** Costos iniciales de adquisición o configuración de servidores, o costos recurrentes si se opta por servicios en la nube.

Costos Operativos (Post-Implementación)

- Mantenimiento del software y la base de datos.
- Soporte técnico a usuarios.
- Costos continuos de infraestructura (energía, conectividad, cloud si aplica).
- Capacitación continua.

Retorno de la Inversión (ROI) y Beneficios

Aunque muchos beneficios son cualitativos, su impacto económico es considerable:

- **Reducción de Costos Directos:** Disminución de horas-hombre dedicadas a tareas manuales de gestión de datos, generación de informes y resolución de inconsistencias.
- **Eficiencia Mejorada:** Optimización de procesos que liberan tiempo de personal cualificado para tareas de mayor valor añadido.
- **Mejora en la Toma de Decisiones:** Decisiones basadas en datos precisos y oportunos pueden llevar a una asignación más eficiente de recursos, optimización de programas académicos y mejora en la captación de fondos de investigación.
- **Cumplimiento y Acreditación:** Un sistema robusto facilita el cumplimiento de normativas y los procesos de acreditación, evitando posibles sanciones o asegurando financiamiento.
- **Reducción de Errores:** La automatización y validaciones disminuyen la incidencia de errores costosos.

La inversión inicial se justifica por los ahorros a mediano y largo plazo, la mejora en la calidad de la gestión y el soporte a las funciones críticas de la Universidad. Se recomienda realizar un análisis costo-beneficio más detallado una vez afinados los requerimientos técnicos específicos y la selección de plataformas.

1.7.3. Viabilidad Operativa

El sistema se considera operativamente viable, ya que:

- **Alineación con Necesidades del Usuario:** El sistema se diseñará basándose en los requisitos específicos proporcionados, buscando resolver problemáticas concretas de la gestión académica y de investigación. Las interfaces diferenciadas para usuarios y administradores con distintos niveles de funcionalidad CRUD están pensadas para adaptarse a sus roles.
- **Aceptación por parte de los Usuarios:** Se mitigarán resistencias al cambio mediante:
 - **Capacitación:** Se planificarán sesiones de capacitación para los diferentes perfiles de usuario.
 - **Diseño Intuitivo:** Se priorizará una interfaz de usuario clara, amigable y fácil de usar.
 - **Participación:** Involucrar a representantes de los usuarios finales durante las fases de diseño y pruebas de aceptación (UAT) para asegurar que el sistema cumpla con sus expectativas y facilite su trabajo diario.
- **Integración en Flujos de Trabajo:** El sistema está concebido para integrarse y optimizar los flujos de trabajo existentes. Es posible que se requiera una adaptación de ciertos procesos manuales para aprovechar al máximo las capacidades del nuevo sistema, lo cual se gestionará como parte del proceso de implementación.
- **Soporte y Mantenimiento:** Se deberá establecer un plan de soporte técnico continuo (interno o externo) para resolver incidencias, realizar mantenimiento preventivo y aplicar futuras actualizaciones o mejoras. La documentación técnica y de usuario generada facilitará estas labores.
- **Seguridad de la Información:** Se implementarán medidas de seguridad adecuadas para proteger la integridad, confidencialidad y disponibilidad de los datos, incluyendo gestión de accesos basada en roles, protección contra amenazas comunes y políticas de respaldo.

1.7.4. Viabilidad de Plazos (Cronograma – 15 Meses)

Un plazo total de 15 meses de trabajo efectivo se considera factible para la realización de este proyecto, dada su envergadura y la adopción de la metodología Scrum. Este marco de trabajo ágil permitirá entregas incrementales y una adaptación continua a lo largo del desarrollo.

La planificación temporal se estructura de la siguiente manera, considerando los períodos de receso académico:

- **Inicio del Proyecto:** Septiembre 2025.
- **Duración Total de Trabajo Efectivo:** 15 meses.
- **Meses No Laborables (Receso Académico):** Febrero 2026 y Agosto 2026.
- **Finalización Estimada del Proyecto:** Enero 2027.

El desarrollo se organizará en Sprints (ciclos de trabajo cortos, por ejemplo, de 4 semanas). Los objetivos específicos se distribuirán en bloques de trabajo de la siguiente forma:

- **Bloque 1: Objetivo Específico 1 (Formulación y Modelado Conceptual).**
 - **Período de Trabajo:** Septiembre 2025 – Enero 2026 (5 meses de trabajo).
 - **Sprints Estimados:** Aproximadamente 5 Sprints.
 - **Enfoque Principal:** Dedicado a la elaboración del informe técnico inicial (incluyendo análisis de viabilidad, justificación, etc.), análisis exhaustivo de requisitos, y el diseño detallado del Modelo Entidad-Relación (MER) y el Modelo Conceptual de la base de datos.
- **Bloque 2: Objetivo Específico 2 (Diseño e Implementación de la Base de Datos).**
 - **Período de Trabajo:** Marzo 2026 – Julio 2026 (5 meses de trabajo, posterior al receso de Febrero 2026).
 - **Sprints Estimados:** Aproximadamente 5 Sprints.
 - **Enfoque Principal:** Centrado en la transformación del modelo conceptual al modelo relacional, aplicación de técnicas de normalización (hasta 3FN), escritura de los scripts DDL para la creación de tablas y restricciones, carga de datos de prueba, y la implementación de índices y vistas predefinidas para optimizar consultas.
- **Bloque 3: Objetivos Específicos 3 y 4 (Diseño y Desarrollo del Sistema Web, Integración con la Base de Datos, Procesos CRUD, Pruebas y Validación).**
 - **Período de Trabajo:** Septiembre 2026 – Enero 2027 (5 meses de trabajo, posterior al receso de Agosto 2026).
 - **Sprints Estimados:** Aproximadamente 5 Sprints.
 - **Enfoque Principal:** Esta fase comprenderá el diseño de maquetas del sistema web, el desarrollo frontend (HTML, CSS) y backend (PHP), la conexión con la base de datos implementada, y la programación de las funcionalidades CRUD. De forma paralela e integrada en los Sprints, se llevará a cabo el Objetivo 4, realizando pruebas unitarias, de integración, de sistema, y las pruebas de aceptación del usuario (UAT). Se finalizará con la corrección de errores, la elaboración de la documentación final y la preparación para el despliegue.

Consideraciones para cumplir el plazo: El cumplimiento de este cronograma dependerá de varios factores críticos:

- **Gestión de Riesgos Eficaz:** Identificación temprana y mitigación proactiva de posibles riesgos, como cambios no controlados en el alcance (scope creep), retrasos en la toma de decisiones o validaciones por parte del “cliente” (en este caso, el profesorado del curso), o la aparición de problemas técnicos imprevistos.
- **Dedicación y Compromiso del Equipo:** Es fundamental contar con la dedicación adecuada del equipo de desarrollo estudiantil y un compromiso con los objetivos de cada Sprint y las entregas planificadas.
- **Aplicación Rigurosa de Scrum:** La correcta implementación de las ceremonias (Sprint Planning, Daily Scrum, Sprint Review, Sprint Retrospective) y artefactos (Product Backlog, Sprint Backlog, Incremento) de Scrum facilitará la transparencia, la inspección y la adaptación continua, elementos clave para el manejo eficiente del tiempo.
- **Comunicación Constante y Fluida:** Mantener una comunicación efectiva dentro del equipo de desarrollo y con el “cliente” para asegurar la alineación y la rápida resolución de dudas o problemas que puedan surgir.

En conclusión, el plazo de 15 meses de trabajo efectivo, distribuido según el calendario propuesto y gestionado mediante la metodología ágil Scrum, ofrece un marco temporal realista y estructurado para la finalización exitosa del proyecto por parte del equipo de estudiantes.

1.8. Metodología de Desarrollo Propuesta: Scrum

Para un equipo de 6 estudiantes de Ingeniería Civil en Informática que están comenzando a conocer las metodologías de desarrollo de software y que tienen un proyecto de 15 meses, la metodología más apropiada sería **Scrum**.

Justificación de la Elección de Scrum

Scrum es un marco de trabajo ágil que se adapta muy bien a las características de este proyecto y del equipo por las siguientes razones:

1. **Desarrollo Iterativo e Incremental:** Scrum divide el proyecto en ciclos cortos llamados *Sprints* (generalmente de 2 a 4 semanas). Al final de cada Sprint, el equipo entrega un incremento funcional del producto. Esto es ideal para un proyecto de 15 meses, ya que permite al equipo mostrar progreso tangible de forma regular, obtener retroalimentación temprana del “cliente” (profesor del curso) y ajustar el rumbo si es necesario. Para estudiantes, ver resultados funcionales de forma periódica es altamente motivador.

2. **Adaptabilidad y Flexibilidad:** Aunque los requisitos iniciales están definidos, es probable que surjan nuevos entendimientos o se necesiten ajustes, especialmente en la interfaz de usuario y la experiencia de usuario (UI/UX). Scrum permite la adaptación a cambios entre Sprints, incorporando nuevos requisitos o modificaciones en el *Product Backlog*.
3. **Estructura y Claridad para Equipos Nuevos:** Para estudiantes que recién se inician en metodologías, Scrum ofrece un marco con roles, eventos y artefactos definidos que proporcionan una estructura clara:

- **Roles Adaptados:** Para el contexto estudiantil, estos roles pueden ser:
 - **Product Owner (Dueño del Producto):** Rol asumido idealmente por el “cliente” (profesor) o, de forma interna y en representación del cliente, por un miembro del equipo (ej. el Jefe de Proyecto, Javier Rojas) encargado de definir y priorizar los ítems del Product Backlog.
 - **Scrum Master (Facilitador):** Rol que podría ser asumido por el Jefe de Proyecto (Javier Rojas), enfocándose en asegurar que el equipo siga las prácticas de Scrum, facilitar las reuniones y eliminar impedimentos.
 - **Development Team (Equipo de Desarrollo):** Los 6 estudiantes, como un equipo auto-organizado y multidisciplinario, responsable de entregar el incremento del producto.
- **Eventos (Reuniones):**
 - *Sprint Planning:* Planificación del trabajo a realizar en el Sprint.
 - *Daily Scrum:* Reunión diaria corta para sincronizar al equipo.
 - *Sprint Review:* Demostración del incremento del producto y recolección de feedback.
 - *Sprint Retrospective:* Reflexión sobre el Sprint anterior para identificar mejoras en el proceso.

Estas reuniones estructuradas ayudan a mantener el enfoque y la comunicación.

- **Artefactos:**
 - *Product Backlog:* Lista priorizada de todas las funcionalidades y requisitos del proyecto.
 - *Sprint Backlog:* Conjunto de ítems del Product Backlog seleccionados para un Sprint, más el plan para entregarlos.
 - *Incremento del Producto:* La suma de todos los ítems del Product Backlog completados durante un Sprint y Sprints anteriores, que debe ser potencialmente entregable.

Esta estructura es fundamental para guiar al equipo a lo largo de los 15 meses de proyecto.

4. **Fomenta la Colaboración y Comunicación:** El tamaño del equipo (6 personas) es ideal para Scrum. Las reuniones como el Daily Scrum y la planificación y revisión conjuntas de

los Sprints aseguran que todos los miembros estén alineados, compartan información y colaboren estrechamente para alcanzar el objetivo del Sprint.

5. **Gestión de Riesgos Temprana:** La naturaleza iterativa y las revisiones constantes (Sprint Review, Sprint Retrospective) permiten identificar problemas, riesgos o malentendidos de forma temprana, en lugar de esperar hasta el final del proyecto. Esto es crucial en un proyecto de 15 meses, donde los problemas no detectados pueden escalar significativamente.
6. **Enfoque en la Entrega de Valor:** Cada Sprint se enfoca en entregar las funcionalidades de mayor valor para el cliente (priorizadas en el Product Backlog), lo que asegura que el sistema evolucione de manera útil y relevante desde las primeras etapas.
7. **Oportunidad de Aprendizaje Práctico:** Implementar Scrum proporcionará al equipo una valiosa experiencia práctica con una de las metodologías ágiles más utilizadas en la industria del software. Esto constituye un objetivo de aprendizaje importante en su formación como Ingenieros Civiles en Informática, preparándolos para entornos de trabajo reales.

Aunque otras metodologías ágiles como Kanban podrían ser consideradas por su simplicidad visual y enfoque en el flujo, la estructura de Sprints de Scrum, junto con sus roles y ceremonias definidas, ofrece un andamiaje más robusto y guiado para un equipo de estudiantes que se enfrenta a un proyecto de duración considerable y que necesita aprender a gestionar su trabajo de forma sistemática, colaborativa y adaptativa. Les ayudará a mantener un ritmo sostenible y a asegurar entregas consistentes y de valor a lo largo de los 15 meses del proyecto.

1.9. Planificación Temporal del Desarrollo del Proyecto (Metodología Scrum)

El marco ágil Scrum estructura el avance en iteraciones, ajustando los incrementos al ciclo de vida del prototipo[cite: 1]. La calendarización distribuye el esfuerzo a lo largo de quince meses efectivos, omitiendo los recesos académicos y fragmentando el trabajo en bloques orientados a las fases CDI y su consecuente validación funcional.

Consideraciones Metodológicas de la Planificación:

- **Inicio del Proyecto:** Septiembre 2025.
- **Duración Total:** 15 meses de trabajo efectivo (excluyendo recesos de Febrero y Agosto 2026).
- **Iteraciones:** Sprints de 4 semanas, subordinados al refinamiento continuo del *Product Backlog*.
- **Delimitación del Alcance:** Las entregas finales se evalúan bajo métricas de prototipado, descartando actividades de despliegue operativo comercial.

Tabla 1.2: Planificación Temporal del Proyecto Orientado a Prototipo

Fase / Bloque de Objetivos	Mes de Trabajo	Mes y Año Calendario	Sprint #	Duración Estimada	Foco Principal y Actividades Clave	Entregables Clave del Sprint
Bloque 1: Fase de Concepción (Obj. Específico 1)	1	Sep 2025	1	4 semanas	Configuración del entorno Scrum. Levantamiento de requerimientos funcionales y extracción de entidades del dominio académico.	Acta de constitución, <i>Product Backlog</i> inicial, listado de requerimientos y entidades.
Bloque 1: Fase de Concepción (Obj. Específico 1)	2	Oct 2025	2	4 semanas	Estructuración del informe técnico preliminar. Evaluación de factibilidad técnica para el desarrollo web.	Borrador de informe técnico, matriz de viabilidad.
Bloque 1: Fase de Concepción (Obj. Específico 1)	3	Nov 2025	3	4 semanas	Definición exhaustiva de atributos y restricciones. Inicio del diseño del Modelo Entidad-Relación (MER).	Especificación de atributos, borrador inicial del MER.
Bloque 1: Fase de Concepción (Obj. Específico 1)	4	Dic 2025	4	4 semanas	Refinamiento del diagrama conceptual. Consolidación de cardinalidades (1:N, M:N).	Modelo Entidad-Relación finalizado.
Bloque 1: Fase de Concepción (Obj. Específico 1)	5	Ene 2026	5	4 semanas	Validación del modelo conceptual con contraparte académica. Revisión final del informe técnico.	Documento de formulación aprobado.
<i>Receso Académico</i>	–	<i>Feb 2026</i>	–	–	<i>Suspensión temporal de actividades</i>	–

Continuará en la página siguiente...

Tabla 1.2: Planificación Temporal del Proyecto (Continuación)

Fase / Bloque de Objetivos	Mes de Trabajo	Mes y Año Calendario	Sprint #	Duración Estimada	Foco Principal y Actividades Clave	Entregables Clave del Sprint
Bloque 2: Fase de Diseño (Obj. Específico 2)	6	Mar 2026	6	4 semanas	Transformación del MER al modelo relacional. Aplicación de normalización hasta Primera Forma Normal (1FN).	Modelo relacional preliminar.
Bloque 2: Fase de Diseño (Obj. Específico 2)	7	Abr 2026	7	4 semanas	Culminación de normalización estructural (3FN). Elaboración de wireframes de bajo nivel para interfaces.	Esquemas en 3FN, wireframes de estructura base.
Bloque 2: Fase de Diseño (Obj. Específico 2)	8	May 2026	8	4 semanas	Diseño de mockups de alta fidelidad diferenciando perfiles (administrador/limitado). Arquitectura de datos final.	Interfaces gráficas conceptuales, arquitectura de software.
Bloque 3: Fase de Implementación (Obj. Específico 3)	9	Jun 2026	9	4 semanas	Escritura de scripts DDL. Creación física de tablas y llaves foráneas. Poblamiento con <i>mock data</i> .	Base de datos estructurada y con datos de prueba.
Bloque 3: Fase de Implementación (Obj. Específico 3)	10	Jul 2026	10	4 semanas	Desarrollo de maquetación HTML5 y reglas CSS3. Configuración del servidor de desarrollo controlado.	Vistas frontend estáticas operativas.
<i>Receso Académico</i>	–	<i>Ago 2026</i>	–	–	<i>Suspensión temporal de actividades</i>	–
Continuará en la página siguiente...						

Tabla 1.2: Planificación Temporal del Proyecto (Continuación)

Fase / Bloque de Objetivos	Mes de Trabajo	Mes y Año Calendario	Sprint #	Duración Estimada	Foco Principal y Actividades Clave	Entregables Clave del Sprint
Bloque 3: Fase de Implementación (Obj. Específico 3)	11	Sep 2026	11	4 semanas	Programación de conectores PDO/PHP. Implementación de operaciones CRUD para entidades primarias (Profesores, Alumnos).	Módulos CRUD funcionales integrados a la BD.
Bloque 3: Fase de Implementación (Obj. Específico 3)	12	Oct 2026	12	4 semanas	Desarrollo de lógica de negocio compleja (asignaciones a proyectos). Implementación de <i>triggers</i> en el SGBD.	Prototipo web funcional con automatizaciones de motor.
Bloque 4: Fase de Validación (Obj. Específico 4)	13	Nov 2026	13	4 semanas	Diseño de plan de pruebas. Ejecución de pruebas unitarias sobre el código PHP y funciones de motor.	Informes de cobertura de pruebas unitarias.
Bloque 4: Fase de Validación (Obj. Específico 4)	14	Dic 2026	14	4 semanas	Comprobación de integración Frontend-Backend. Auditoría de vulnerabilidades (XSS, SQLi) en entorno simulado.	Reporte de integración y mitigación de fallas de seguridad.
Bloque 4: Fase de Validación (Obj. Específico 4)	15	Ene 2027	15	4 semanas	Refinamiento del prototipo. Evaluación final de rendimiento con índices. Documentación de cierre del sistema.	Prototipo validado empíricamente, Manuales técnicos.

1.10. Diagramas UML

1.10.1. Diagrama de Casos de Uso (Interacción de Usuarios)

El diagrama de la Figura 1.2 modela las interacciones principales de los dos actores centrales del sistema: el Profesor y el Alumno. Las funcionalidades reflejan directamente las reglas de negocio establecidas, como la dirección de proyectos, la supervisión de investigación y la jerarquía de asesorías entre pares y son descritas en la Tabla 1.3.

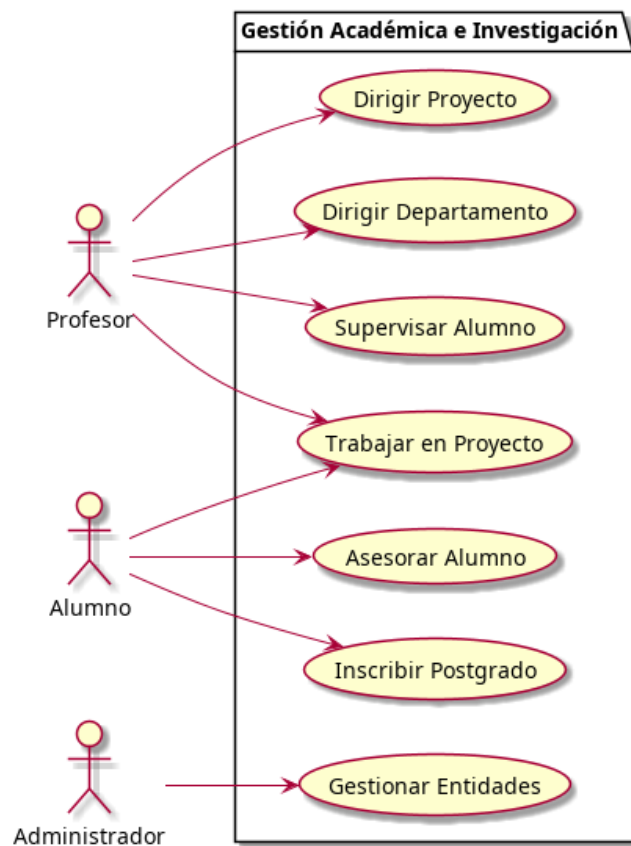


Figura 1.2: Caso de Uso interacción Profesor y Alumno

Justificación:

- Profesor: Puede dirigir varios proyectos (relación N:M *Dirigir*), trabajar en ellos, administrar un departamento (relación 1:1 *Administrar*) y supervisar el trabajo de los alumnos.
- Alumno: Puede trabajar en proyectos de investigación, pertenecer a un programa de postgrado (relación 1:N *Cursar*) y asumir el rol de asesor para orientar a un alumno menos veterano (relación reflexiva 1:N *Asesorar*).

Caso de Uso: Asignar Alumno y Supervisor a Proyecto	
Actores	Administrador del Sistema, Profesor (Director del Proyecto)
Precondiciones	El proyecto se encuentra en estado de ejecución (su atributo <i>FechaFin</i> posee un valor NULL). El alumno y el profesor existen previamente en los registros relacionales.
Flujo Normal de Eventos	
Acción del Actor	Respuesta del Sistema
1. Selecciona la opción para gestionar los participantes de un proyecto de investigación específico.	2. Despliega la interfaz de administración, listando los estudiantes vinculados y el personal académico disponible.
3. Ingresa el identificador (RUN) del alumno que se integrará al equipo de trabajo.	4. Verifica en la tabla Trabajar que el alumno no esté previamente registrado en el proyecto[cite: 4]. Habilita el módulo de selección de supervisor.
5. Asigna el RUN del profesor que ejercerá el rol de supervisor directo sobre las actividades del estudiante.	6. Valida la integridad referencial de los datos. Ejecuta una transacción atómica que inserta la vinculación en la tabla Trabajar y registra la tutela en la tabla Supervisar[cite: 4].
7. Confirma la operación de almacenamiento.	8. Confirma la persistencia de los datos mediante un mensaje de éxito y refresca la vista del proyecto.
Flujos Alternativos	
4a. Duplicidad de participante: Si el motor de base de datos detecta que el alumno ya integra el proyecto, el sistema aborta el flujo y despliega una advertencia. 6a. Violación de restricción funcional: Si se intenta registrar un segundo supervisor para el mismo alumno dentro del mismo proyecto, el SGBD rechaza la inserción mediante la llave primaria compuesta (<i>Proyecto_id</i> , <i>Alumno_id</i>) de la tabla Supervisar, garantizando la regla de negocio institucional[cite: 4].	
Postcondiciones	La base de datos consolida la nueva relación ternaria, manteniendo la coherencia de las dependencias funcionales sin anomalías de inserción.

Tabla 1.3: Especificación del Caso de Uso para la asignación controlada de alumnos y supervisores en investigaciones.

1.10.2. Diagrama de Actividades (Asignación a Proyecto)

El diagrama de la Figura 1.3 detalla el flujo lógico que el sistema debe seguir al incorporar un alumno de postgrado a un proyecto de investigación. Este proceso es crítico porque la base de datos exige que todo alumno que trabaje en un proyecto cuente obligatoriamente con un profesor supervisor exclusivo para ese contexto.

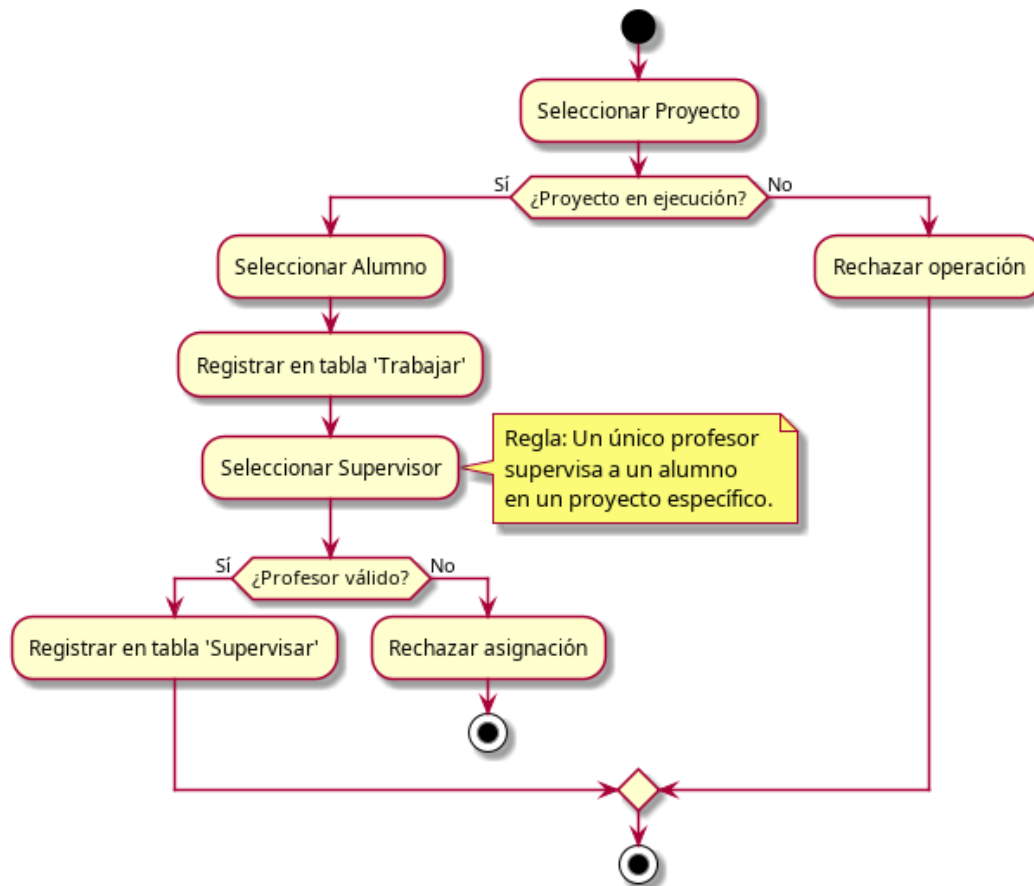


Figura 1.3: Diagrama de actividad de incorporar un alumno de postgrado a un proyecto

Justificación:

- El flujo aborda la relación ternaria *Supervisar* (Proyecto, Alumno, Profesor), asegurando que se respete la dependencia funcional donde un par (Alumno, Proyecto) determina a un único Profesor supervisor.
- Se asegura la inserción transaccional tanto en la tabla *Trabajar* (vinculación al proyecto) como en la tabla *Supervisar* (asignación de tutela).

1.10.3. Diagrama de Estados (Ciclo de Vida de un Proyecto)

El diagrama de la Figura 1.4 representa los diferentes estados por los que transita la entidad Proyecto dentro de la base de datos. Los estados están directamente condicionados por las fechas de inicio y finalización, así como por la asignación presupuestaria.

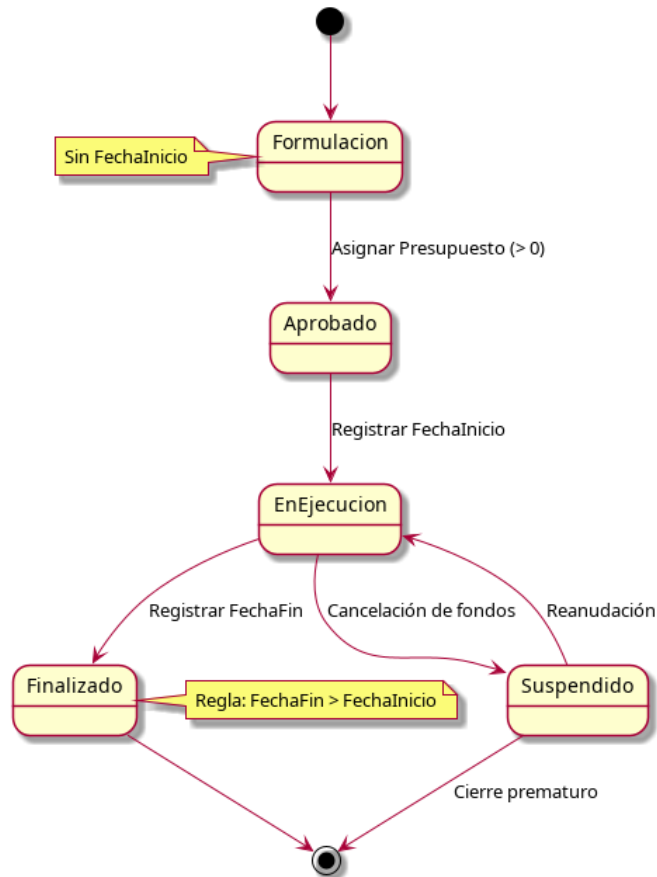


Figura 1.4: Estado de la entidad proyectos

Justificación:

- En Formulaci3n / Aprobado: El sistema exige mediante restricciones (CHECK) que el presupuesto no sea un valor negativo y que la fecha de inicio no pueda evadir la l3gica temporal del servidor (aunque la restricci3n de fecha actual fue eliminada para permitir datos hist3ricos).
- En Ejecuci3n vs Finalizado: Un proyecto se considera activo ("En Ejecuci3n") mientras el atributo FechaFin tenga valor NULL. Una vez que se registra una FechaFin, el sistema valida estructuralmente que esta sea estrictamente posterior a la FechaInicio.

Capítulo 2

Modelo de Datos

2.1. Requisitos

A continuación se establecen los requisitos del sistema de datos solicitado por la universidad.

1. Cada profesor tiene RUN, nombre, edad, rango y especialidad de investigación.
2. Cada proyecto tiene número de proyecto, nombre de patrocinador (por ejemplo, el CSIC), fecha de comienzo, fecha de finalización y presupuesto.
3. Cada alumno tiene RUN, nombre, edad.
4. Cada alumno puede o no pertenecer a un programa de postgrado (por ejemplo, magíster o doctorado), del cual interesa el nombre y código.
5. Cada proyecto está dirigido por un profesor (conocido como investigador principal de ese proyecto).
6. En cada proyecto trabajan uno o varios profesores (conocidos como investigadores de ese proyecto).
7. Cada profesor puede dirigir varios proyectos o trabajar en ellos.
8. En cada proyecto trabajan uno o varios alumnos de postgrado (conocidos como ayudantes de investigación de ese proyecto).
9. Cuando los alumnos de postgrado trabajan en un proyecto, su trabajo debe supervisarlo un profesor. Cada alumno de postgrado puede trabajar en varios proyectos, en cuyo caso puede tener un supervisor diferente en cada uno de ellos.
10. Cada departamento tiene número de departamento, nombre de departamento y despacho principal.

11. Cada departamento tiene un profesor (conocido como director) que lo dirige.
12. Cada profesor trabaja en uno o varios departamentos, y por cada departamento en que trabaja se asocia un porcentaje de tiempo a su trabajo.
13. Cada alumno de postgrado tiene un departamento principal en el que trabaja en su titulación.
14. Cada alumno de postgrado tiene otro alumno de postgrado más veterano (conocido como asesor del alumno) que lo aconseja sobre las asignaturas en las que debe matricularse.

2.2. Modelo Entidad Relación

2.2.1. Definición de Entidades

Herencia

A partir de los requisitos expuestos en las líneas 1 y 2, se identifican las entidades *Profesor* y *Alumno*. Ambas comparten atributos inherentes al concepto de *Persona*, por lo que se propone utilizar una jerarquía de herencia para optimizar el modelo de datos, como se ilustra en la Figura 2.1. Dado que los únicos roles definidos en el sistema son *Profesor* y *Alumno*, y no se contempla la posibilidad de que un individuo ocupe ambos roles simultáneamente, se define esta especialización como **total y sin solapamiento**. Esto implica que cada instancia de *Persona* registrada en el sistema será exclusivamente un *Profesor* o un *Alumno*.

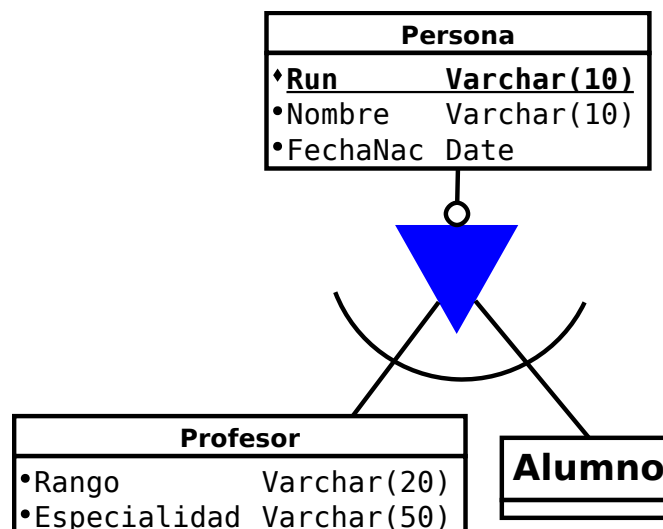


Figura 2.1: Herencia Persona

Entidades individuales

A partir del requisito de la línea 2, se define la entidad *Proyecto*. La estructura detallada de esta entidad, incluyendo sus atributos y características, se presenta en la Tabla 2.1.

Proyecto	
Numero (PK)	Integer
Patrocinador	Varchar (20)
FechaInicio	Date
FechaFin	Date
Presupuesto	Money

Tabla 2.1: Entidad Proyecto

El requerimiento expuesto en la línea 4 da origen a la entidad *Postgrado*, cuya composición y atributos se desglosan en la Tabla 2.2.

Postgrado	
Codigo (PK)	Integer
Nombre	Varchar (30)

Tabla 2.2: Entidad Postgrado

La entidad *Departamento*, identificada a partir del requerimiento de la línea 10, se detalla en la Tabla 2.3, donde se especifican sus atributos y características.

Departamento	
Numero (PK)	Integer
Nombre	Varchar (30)
Despacho	Integer

Tabla 2.3: Entidad Departamento

2.2.2. Definición de Relaciones

Relaciones n-aria

A partir de los requisitos de las líneas 6 y 9, se define la relación ternaria *Supervisar* que vincula a *Profesor*, *Alumno* y *Proyecto* (ver Figura 2.2). Aunque un profesor puede supervisar a varios alumnos y un proyecto puede involucrar a muchos, el requisito 9 impone una restricción clave: la supervisión de un alumno en un proyecto específico es realizada por un único profesor. Esto refina la cardinalidad general, estableciendo una dependencia funcional donde el par (*Alumno*, *Proyecto*) determina al *Profesor* supervisor.

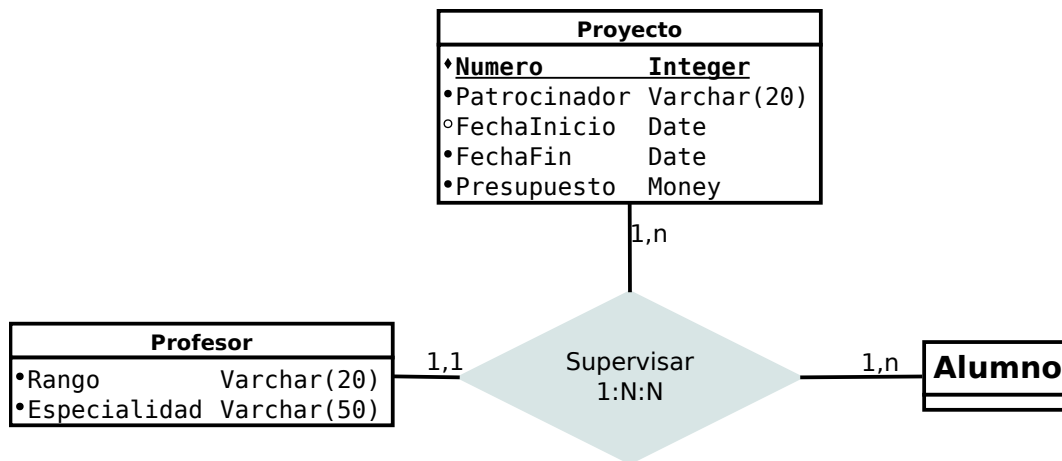


Figura 2.2: Relación Supervisar

Relaciones Muchos a Muchos (N:N)

Si bien el requisito de la línea 5 sugiere que un proyecto es dirigido por 'un' profesor, para dotar al sistema de mayor flexibilidad y capacidad a futuro, se opta por modelar la relación *Dirigir* como muchos a muchos (N:M). Esta decisión de diseño, ilustrada en la Figura 2.3, permite que el sistema registre múltiples directores o co-directores por proyecto si fuera necesario, cumpliendo a la vez con el requisito de la línea 7 que establece que un profesor puede dirigir varios proyectos. Este enfoque proporciona mayor adaptabilidad a casos de uso complejos.



Figura 2.3: Relación Dirigir

El requisito de la línea 12 establece que un profesor puede trabajar en varios departamentos y que a esta vinculación se le asocia un "porcentaje de tiempo". Para modelar esto correctamente, se define la relación *Laborar* con una cardinalidad de muchos a muchos (N:M) entre *Profesor* y *Departamento*. Esta relación, ilustrada en la Figura 2.4, incluye el atributo *porcentaje* que depende de la combinación de un profesor y un departamento específicos.

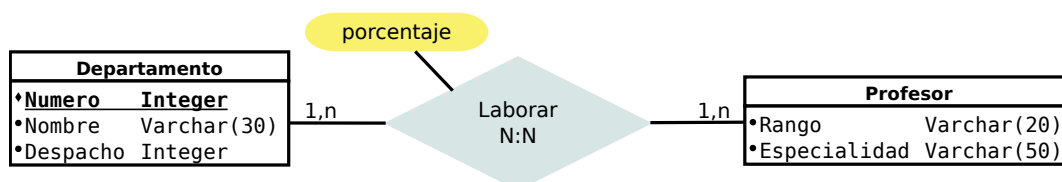


Figura 2.4: Relación Laborar

A partir del requisito expuesto en la línea 8, se establece la relación Trabajar entre las entidades Proyecto y Alumno. Se trata de una relación con cardinalidad muchos a muchos (N:M), ya que un proyecto puede tener “uno o varios” alumnos trabajando en él, y (como se complementa en la línea 9) un alumno puede trabajar en varios proyectos (Ver Figura 2.5).

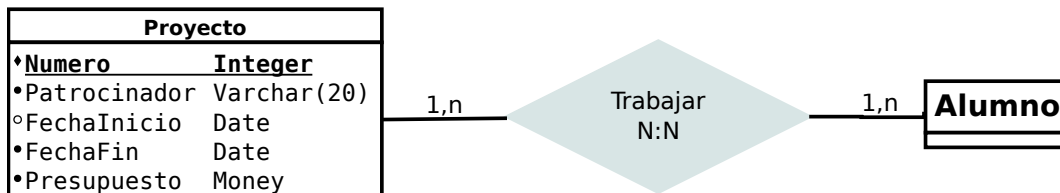


Figura 2.5: Relación Trabajar

Relaciones Uno a Muchos (1:N)

La relación Cursar, que vincula a un Alumno con un Postgrado, se deriva del requisito de la línea 4. Este define una cardinalidad de uno a muchos (1:N), donde un Postgrado puede tener múltiples alumnos inscritos, pero un Alumno solo puede pertenecer a un programa de postgrado a la vez, siendo esta participación opcional. La Figura 2.6 ilustra esta relación y sus respectivas cardinalidades (0,1) para Alumno y (0,N) para Postgrado.

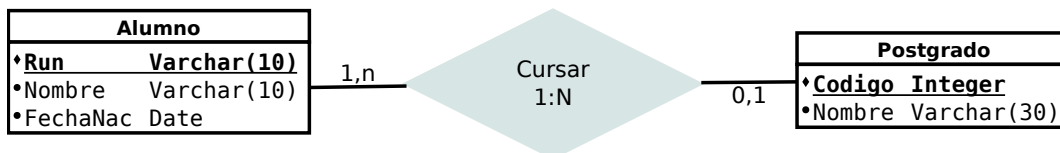


Figura 2.6: Relación Cursar

A partir del requisito de la línea 13, se define la relación Pertenece como una asociación de uno a muchos (1:N) que vincula a Departamento con Alumno. Como se ilustra en la Figura ??, esta cardinalidad refleja que, si bien un departamento puede tener múltiples alumnos (1,N), cada alumno de postgrado debe estar asociado obligatoriamente a un único departamento principal (1,1).

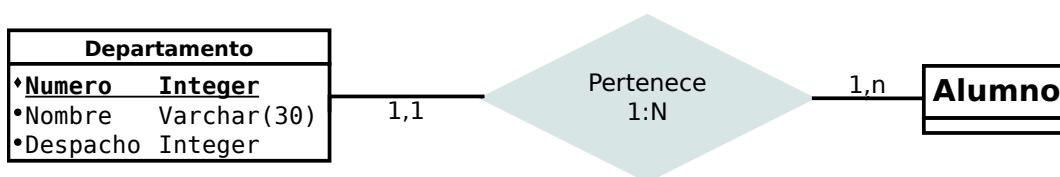


Figura 2.7: Relación Pertenece

Relaciones Uno a Uno (1:1)

A partir del requisito de la línea 11, se define la relación Administrar como una asociación de uno a uno (1:1) que vincula a un Departamento con su Profesor director. La Figura 2.8 ilustra esta estructura, donde la participación del Departamento es obligatoria y única (1,1), mientras que la del Profesor en este rol es opcional y, como máximo, única (0,1).



Figura 2.8: Relación Administrar

Relaciones Reflexivas

El requisito de la línea 14, que establece que cada alumno tiene un asesor más veterano, da origen a la relación reflexiva Asesorar en la entidad Alumno. Se trata de una relación de uno a muchos (1:N), donde un alumno en el rol de "asesor" puede guiar a múltiples alumnos "asesorados", mientras que cada asesorado está vinculado obligatoriamente a un único asesor. Esta estructura recursiva se representa en la Figura 2.9.

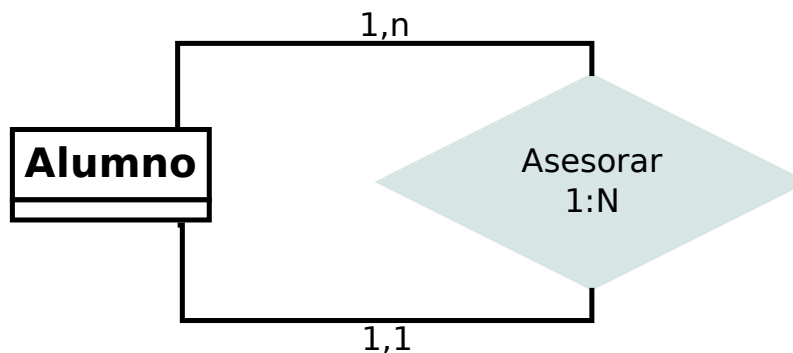


Figura 2.9: Relación Asesorar

2.2.3. Modelo Entidad Resultante

Una vez establecidas las entidades y sus atributos como se presentó en la sección 2.2.1 y las relaciones y herencia presentadas en la sección 2.2.2, se obtiene el modelo Entidad Relación completo que se ve en la Figura 2.10.

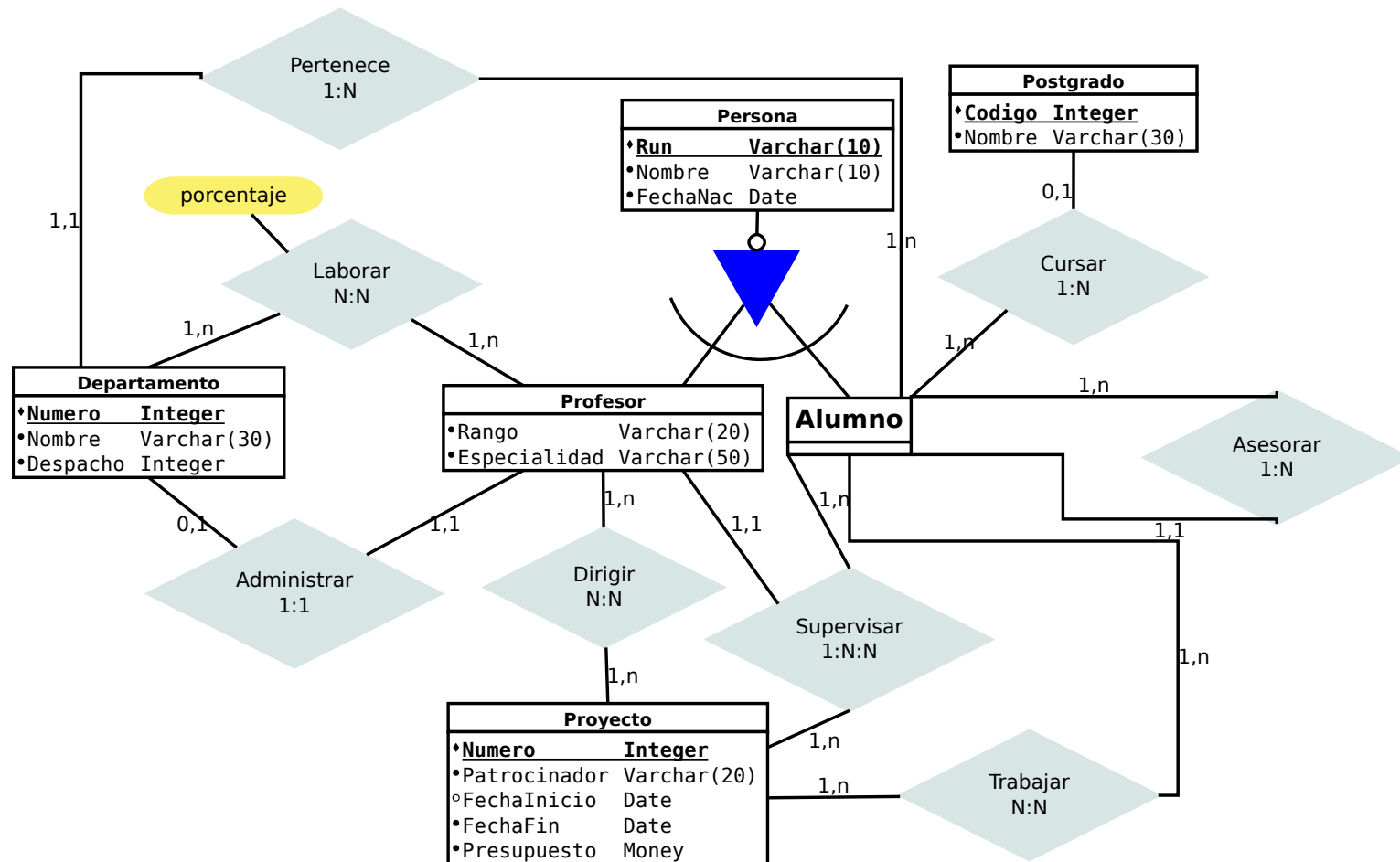


Figura 2.10: Modelo Entidad Relación de la universidad

2.2.4. Modelo Entidad Relación sin Herencia

Considerando el Modelo Entidad Relación de la Figura 2.10 se aprecia que se implementó una herencia total sin solapamiento, lo que implica que para eliminarla se debe hacer eliminación del supertipo, por lo que los atributos del padre como sus relaciones pasan ahora a las entidades hijas, con esto obtenemos el modelo entidad relación final que se ve en la Figura 2.11.

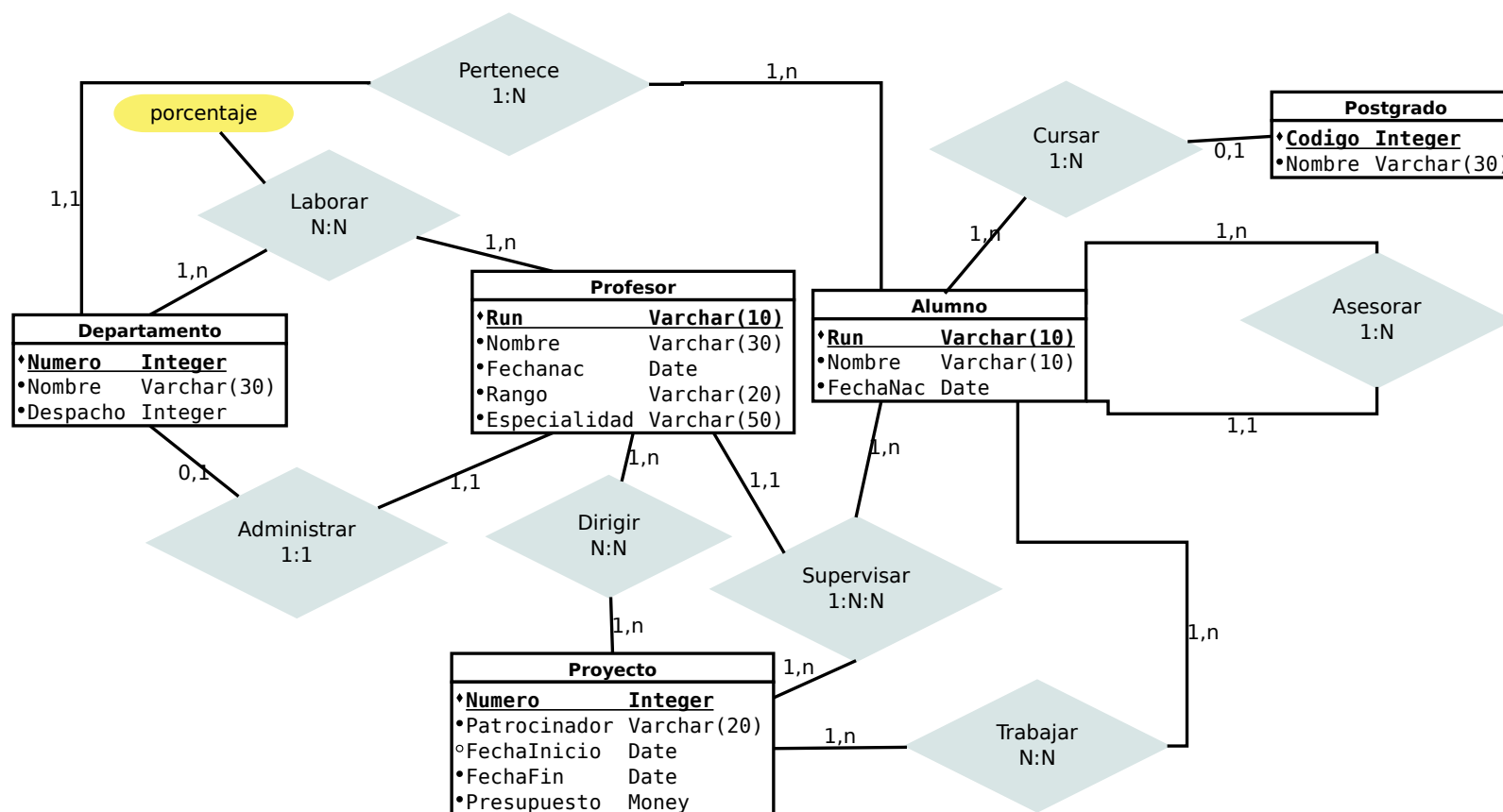


Figura 2.11: MER de la universidad aplicado Eliminación del supertipo a la herencia

2.3. Modelo Relacional

En esta etapa se procede con la conversión del Modelo Entidad-Relación (MER) al Modelo Relacional (MR). El MER de referencia, en el que se ha resuelto la especialización/generalización, se muestra en la Figura 2.11. La siguiente descripción se enfocará exclusivamente en las tablas generadas a partir de relaciones complejas (N:M, ternarias) o aquellas que requirieron modificaciones estructurales. Las entidades que no se detallan a continuación se mapean a tablas de forma directa, conservando su estructura original.

2.3.1. Conversión Relaciones N-arias

Supervisar

Al transformar el Modelo Entidad-Relación (MER) al Modelo Relacional (MR), la relación ternaria Supervisar (Figura 2.12) se convierte en una nueva tabla. Esta tabla, también llamada Supervisar, incluye las claves foráneas de las tres entidades participantes: Proyecto, Alumno y Profesor.

La clave primaria para esta nueva tabla se define a partir de la dependencia funcional del requisito 9: como un par (Alumno, Proyecto) determina a un único Profesor supervisor, la clave primaria de la tabla Supervisar se compone únicamente de las claves foráneas de Alumno y Proyecto. La clave del Profesor, al ser determinada por las otras dos, se mantiene como un atributo no clave. La estructura resultante se detalla en la Tabla 2.4.

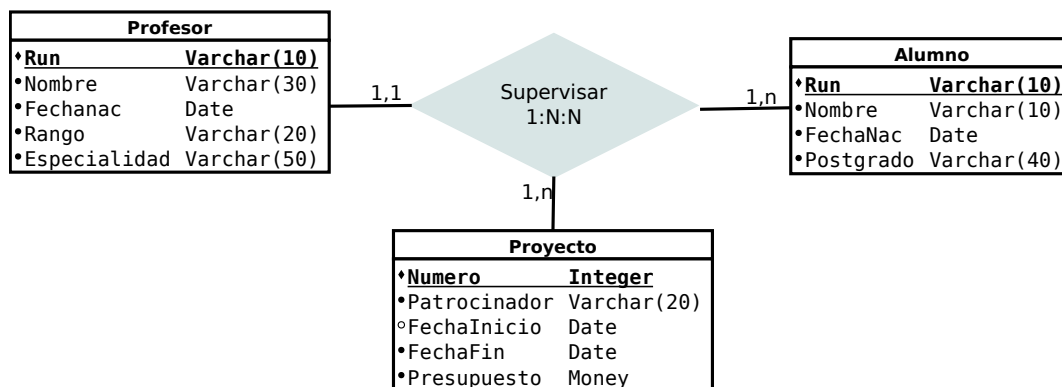


Figura 2.12: Relación Supervisar

Supervisar	
Proyecto_id (PFK)	Integer
Alumno_id (PFK)	Varchar(10)
Profesor_id (FK)	Varchar(10)

Tabla 2.4: MR de la Relación Supervisar

2.3.2. Conversión Relaciones N:N

Dirigir

En el proceso de conversión del Modelo Entidad-Relación (MER) al Modelo Relacional (MR), la relación de muchos a muchos (N:M) *Dirigir*, ilustrada en la Figura 2.13, se transforma en una nueva tabla. Esta tabla, también denominada *Dirigir*, hereda como claves foráneas las claves primarias de las entidades que relaciona (*Proyecto* y *Profesor*), y la combinación de ambas conforma su nueva clave primaria. La estructura final se presenta en la Tabla 2.5.



Figura 2.13: Relación Dirigir

Dirigir	
Proyecto_id (PFK)	Integer
Profesor_id (PFK)	Varchar(10)

Tabla 2.5: MR de la Relación Dirigir

Laborar

En el proceso de conversión del MER al Modelo Relacional, la relación N:M *Laborar* (Figura 2.14) da origen a una nueva tabla. Dicha tabla, llamada *Laborar*, hereda las claves primarias de *Profesor* y *Departamento* para formar su propia clave primaria compuesta. Adicionalmente, se incluye como atributo no clave el *Porcentaje* que pertenecía a la relación. La estructura resultante se detalla en la Tabla 2.6.

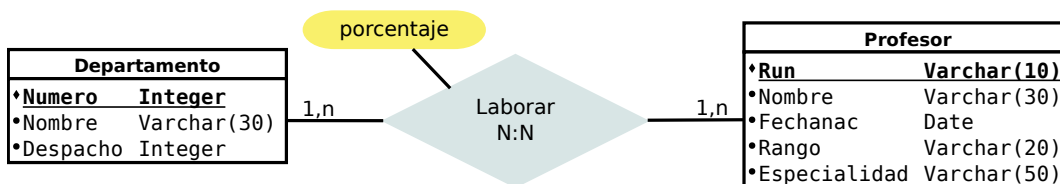


Figura 2.14: Relación Laborar

Laborar	
Departameno_id (PFK)	Integer
Profesor_id (PFK)	Varchar(10)
Porcentaje	Integer

Tabla 2.6: MR de la Relación Laborar

Trabajar

La relación Trabajar, representada en la Figura 2.15, es una relación de cardinalidad muchos a muchos (N:M) entre Proyecto y Alumno. Durante el proceso de conversión del Modelo Entidad-Relación (MER) al Modelo Relacional (MR), este tipo de relación se transforma en una nueva tabla. Para este caso, dicha transformación da origen a la tabla Trabajar, presentada en la Tabla 2.7.

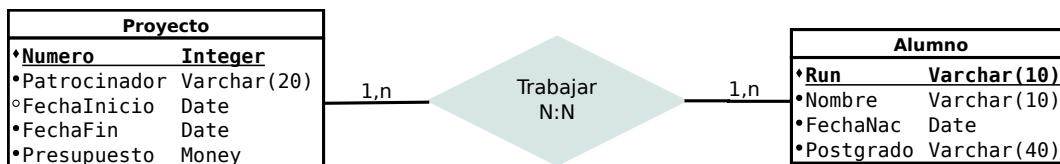


Figura 2.15: Relación Trabajar

Trabajar	
Proyecto_id (PFK)	Integer
Alumno_id (PFK)	Varchar(10)

Tabla 2.7: MR de la Relación Trabajar

2.3.3. Conversión Relaciones 1:N

Cursar

Al transformar el Modelo Entidad-Relación (MER) al Modelo Relacional (MR), la relación de uno a muchos (1:N) Cursar (Figura 2.16) se resuelve siguiendo la regla estándar para este tipo de cardinalidad, la cual no requiere la creación de una tabla nueva para la relación.

El procedimiento correcto consiste en propagar la clave primaria de la entidad del lado "1" de la relación (Postgrado) a la tabla de la entidad del lado "N" (Alumno). Específicamente:

- Se añade una columna de clave foránea en la tabla Alumno que haga referencia a la clave primaria de Postgrado.

- Dado que el requisito de la línea 4 indica que la participación del alumno es opcional ('puede o no pertenecer'), esta nueva columna de clave foránea en la tabla *Alumno* debe permitir valores NULL.

Este enfoque es más eficiente y evita la creación de una tabla adicional innecesaria. La Tabla 2.8 muestra la estructura de la tabla *Alumno* modificada para incluir esta clave foránea.

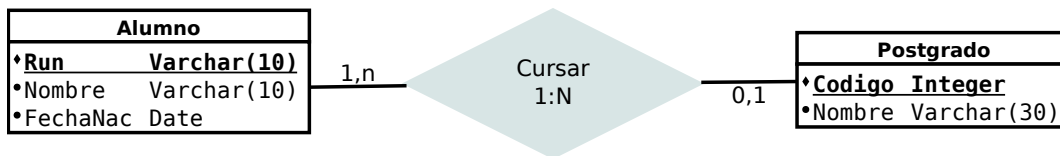


Figura 2.16: Relación Cursar

Alumno	
Run (PK)	Varchar(10)
Nombre	Varchar(40)
Fechanac	Date
<i>Postgrado_id</i> (FK)	Integer

Tabla 2.8: MR de la Relación Cursar

Pertenece

Al transformar la relación de uno a muchos (1:N) *Pertenece* (Figura 2.17) al Modelo Relacional, se aplica la regla de propagación de claves. El procedimiento es el siguiente:

- La clave primaria de la entidad en el lado “uno” de la relación (*Departamento*) se propaga a la tabla de la entidad en el lado “muchos” (*Alumno*).
- La clave primaria de *Departamento* (*Numero*), se añade como clave foránea a la tabla *Alumno*.
- Como la participación del *Alumno* es obligatoria (1,1), según se infiere de la línea 13, estas nuevas columnas en la tabla *Alumno* no deben permitir valores nulos.

La estructura resultante de la tabla *Alumno* se detalla en la Tabla 2.9.

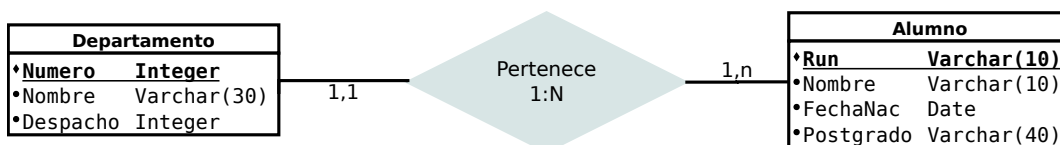


Figura 2.17: Relación Pertenece

Alumno	
Run (PK)	Varchar(10)
Nombre	Varchar(40)
Fechanac	Date
<i>Postgrado_id (FK)</i>	Integer
<i>Departamento_id (FK)</i>	Integer

Tabla 2.9: MR de la Relación Pertenece

2.3.4. Conversión Relaciones 1:1

Administrar

Para convertir la relación de uno a uno (1:1) Administrar (Figura 2.18) al Modelo Relacional, se utiliza el método de propagación de claves. La dirección de esta propagación se decide en función de la participación de cada entidad. Se propaga la clave primaria de la entidad con participación opcional a la tabla de la entidad con participación obligatoria.

- La entidad Departamento tiene participación obligatoria y única (1,1).
- La entidad Profesor tiene participación opcional y única (0,1).

Para optimizar el diseño, la clave primaria de Profesor se añade como clave foránea en la tabla Departamento. Para garantizar la integridad de la relación 1:1, esta nueva columna debe ser NOT NULL y UNIQUE. La estructura resultante se detalla en la Tabla 2.10.



Figura 2.18: Relación Administrar

Departamento	
Numero (PK)	Integer
Nombre	Varchar(30)
Despacho	Integer
<i>Profesor_id (FUK)</i>	Varchar(10)

Tabla 2.10: MR de la Relación Administrar

2.3.5. Conversión Relaciones Reflexivas y otras

Asesorar

A partir del requisito de la línea 14, se define la relación reflexiva (o recursiva) *Asesorar* sobre la propia entidad *Alumno*. Como se ilustra en la Figura 2.19, esta es una relación de uno a muchos (1:N) que distingue los roles de “asesor” y “asesorado”:

- Un *Alumno* (en el rol de asesor) puede asesorar a uno o muchos otros alumnos (1,N).
- Cada *Alumno* (en el rol de asesorado) debe tener exactamente un asesor (1,1).

Para implementar esto en el Modelo Relacional, se añade una clave foránea en la tabla *Alumno* que se referencia a sí misma, como se muestra en la Tabla 2.11.

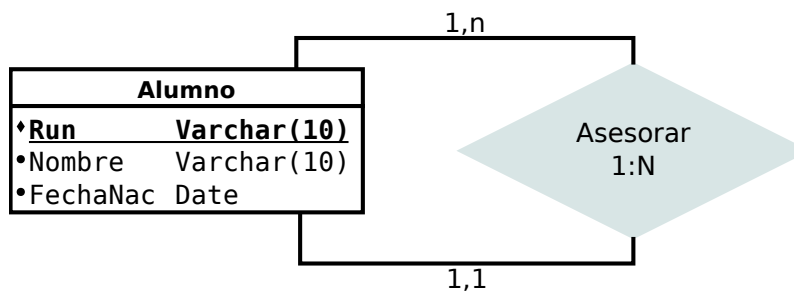


Figura 2.19: Relación Asesorar

Alumno	
Run (PK)	Varchar(10)
Nombre	Varchar(40)
Fechanac	Date
Postgrado_id (FK)	Integer
Departamento_id (FK)	Integer
Alumno_id (FK)	Varchar(10)

Tabla 2.11: MR de la Relación Asesorar

2.3.6. Modelo Relacional Resultante

Una vez realizadas las conversiones presentadas en las secciones 2.3.1-2.3.5 se obtienen las entidades que se ven en la Figura 2.20.

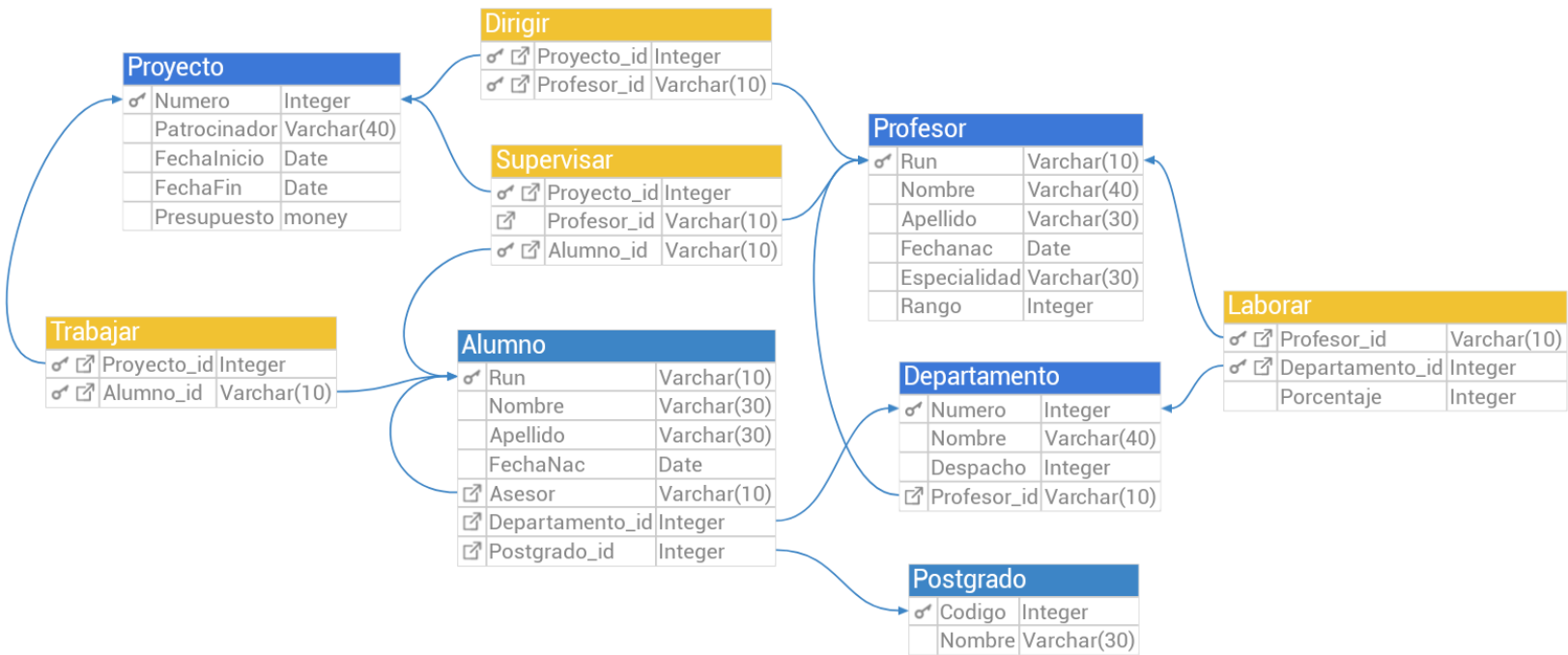


Figura 2.20: Modelo Relacional de la universidad

2.4. Normalización

A continuación, se detalla el análisis de normalización para cada tabla del esquema proporcionado, identificando si se encuentran en Tercera Forma Normal (3NF) o en una forma normal inferior. Se asume que todas las tablas ya cumplen con la Primera Forma Normal (1NF).

2.4.1. Tablas que Cumplen con la Tercera Forma Normal (3NF)

La gran mayoría de las tablas en el esquema proporcionado se encuentran en 3NF. Una tabla está en 3NF si ya está en 2NF y no existen dependencias transitivas (un atributo no-clave no depende de otro atributo no-clave).

- Proyecto:
 - **Clave Primaria (PK):** Numero.
 - **Análisis:** Al tener una clave primaria simple, la tabla está automáticamente en 2NF. No existen dependencias transitivas; los atributos Patrocinador, FechaInicio, FechaFin y Presupuesto dependen directamente del Numero del proyecto. **Se encuentra en 3NF.**
- Postgrado:
 - **PK:**Codigo.
 - **Análisis:** Tiene una clave simple y un único atributo no-clave (Nombre) que depende directamente de ella. **Se encuentra en 3NF.**
- Profesor:
 - **PK:** Run.
 - **Análisis:** Tiene una clave simple, por lo que cumple 2NF. No existen dependencias transitivas entre sus atributos no-clave como Especialidad o Rango. **Se encuentra en 3NF.**
- Dirigir y Trabajar:
 - **Análisis:** Son tablas asociativas puras que solo contienen las claves foráneas que forman su clave primaria. Al no tener atributos no-clave, no pueden tener dependencias parciales ni transitivas. **Se encuentran en 3NF** (y en una forma superior, BCNF).
- Laborar:
 - **PK:** (Profesor_id, Departamento_id).

- **Análisis:** El único atributo no-clave, Porcentaje, depende de la combinación completa de un profesor y un departamento, no de solo uno de ellos, por lo que cumple con 2NF. Al no haber otros atributos no-clave, no pueden existir dependencias transitivas. **Se encuentra en 3NF.**

■ Alumno:

- **PK:** Run.
- **Análisis:** Tiene clave simple, cumpliendo 2NF. Los atributos no-clave, incluyendo las claves foráneas Asesor, Departamento_id y Postgrado_id, son todos características directas del alumno y no dependen entre sí. **Se encuentra en 3NF.**

■ Supervisar:

- **PK:** (Proyecto_id, Alumno_id).
- **Análisis:** El atributo no-clave Profesor_id depende funcionalmente de la clave primaria completa, ya que un alumno en un proyecto específico tiene un único supervisor. Por lo tanto, cumple con 2NF. Al no haber otros atributos no-clave, no es posible una dependencia transitiva. **Se encuentra en 3NF.**

2.4.2. Tablas que NO se Encuentran en Tercera Forma Normal (3NF)

Basado en una interpretación lógica de los atributos, solo una tabla presenta una clara violación a la 3NF.

■ Departamento:

- **PK:** Numero.
- **Atributos no-clave:** Nombre, Despacho, Profesor_id.
- **Análisis de Violación a 3NF:**
 1. La tabla tiene una clave primaria simple (Numero), por lo que cumple con la 2NF.
 2. Sin embargo, existe una **dependencia transitiva** si se asume que el Despacho (oficina) del departamento es en realidad la oficina personal del profesor que lo dirige (Profesor_id).
 3. Bajo esta suposición, tenemos la siguiente cadena de dependencias funcionales:

$$\text{Numero} \rightarrow \text{Profesor_id}$$

$$\text{Profesor_id} \rightarrow \text{Despacho}$$

4. Como el atributo no-clave Despacho depende de otro atributo no-clave (Profesor_id), y no directamente de la clave primaria Numero, la tabla Departamento **no se encuentra en 3NF**. Estaría en 2NF.

Proceso de Normalizar

Para llevar la tabla Departamento de la Figura 2.21a a la Tercera Forma Normal (3NF), debemos eliminar la dependencia transitiva. Esto se logra creando una nueva tabla que contenga el atributo determinante que no es clave principal (Profesor_id) y los atributos que dependen de él (Despacho). Luego, dejamos el atributo determinante en la tabla original como una clave foránea (Foreign Key).

■ Solución: Normalización a 3NF

- Para cumplir con la Tercera Forma Normal (3NF), se debe eliminar la dependencia transitiva separando los atributos correspondientes en una nueva tabla. El atributo Profesor_id se convierte en la clave primaria de esta nueva relación, y se mantiene en la tabla original como clave foránea.

■ Tablas Resultantes en 3NF:

■ Despacho (Nueva tabla para absorber la dependencia transitiva) (Ver Figura 2.21b):

- **PK:** Profesor_id.
- **Atributos no-clave:** Despacho.

■ Departamento (Tabla original ajustada) (Ver Figura 2.21c):

- **PK:** Numero.
- **Atributos no-clave:** Nombre, Profesor_id.
- **FK:** Profesor_id hace referencia a Profesor(Profesor_id).

■ Análisis de Cumplimiento 3NF:

1. En la tabla Despacho, el único atributo no-clave (Despacho) depende directa y completamente de la clave primaria (Profesor_id).
2. En la tabla Departamento, los atributos no-clave (Nombre y Profesor_id) dependen directa y completamente de la clave primaria (Numero).
3. Al separar las tablas, se ha roto la cadena de dependencia $\text{Numero} \rightarrow \text{Profesor_id} \rightarrow \text{Despacho}$. Dado que todos los atributos no-clave ahora dependen únicamente de sus respectivas claves primarias y no existen dependencias transitivas, **ambas relaciones se encuentran en 3NF.**

Departamento		
♂	Numero	Integer
	Nombre	Varchar(40)
	Despacho	Integer
↗	Profesor_id	Varchar(10)

(a) Tabla Departamento Original

Despacho		
♂ ↗	Profesor_id	Varchar(10)
	Despacho	Integer

(b) Tabla Despacho generada por la normalización

Departamento		
♂	Numero	Integer
	Nombre	Varchar(40)
↗	Profesor_id	Varchar(10)

(c) Nueva Tabla Departamento despues de normalizar

Figura 2.21: Normalización Tabla Departamento

2.4.3. Modelo Relacional Normalizado

Considerando las entidades que debieron ser normalizadas de la sección 2.4.2 se obtiene el conjunto de entidades completamente normalizada en 3ra Forma Normal de la Figura 2.22, este nivel de normalización asegura una estabilidad y rapidez en las consultas considerable.

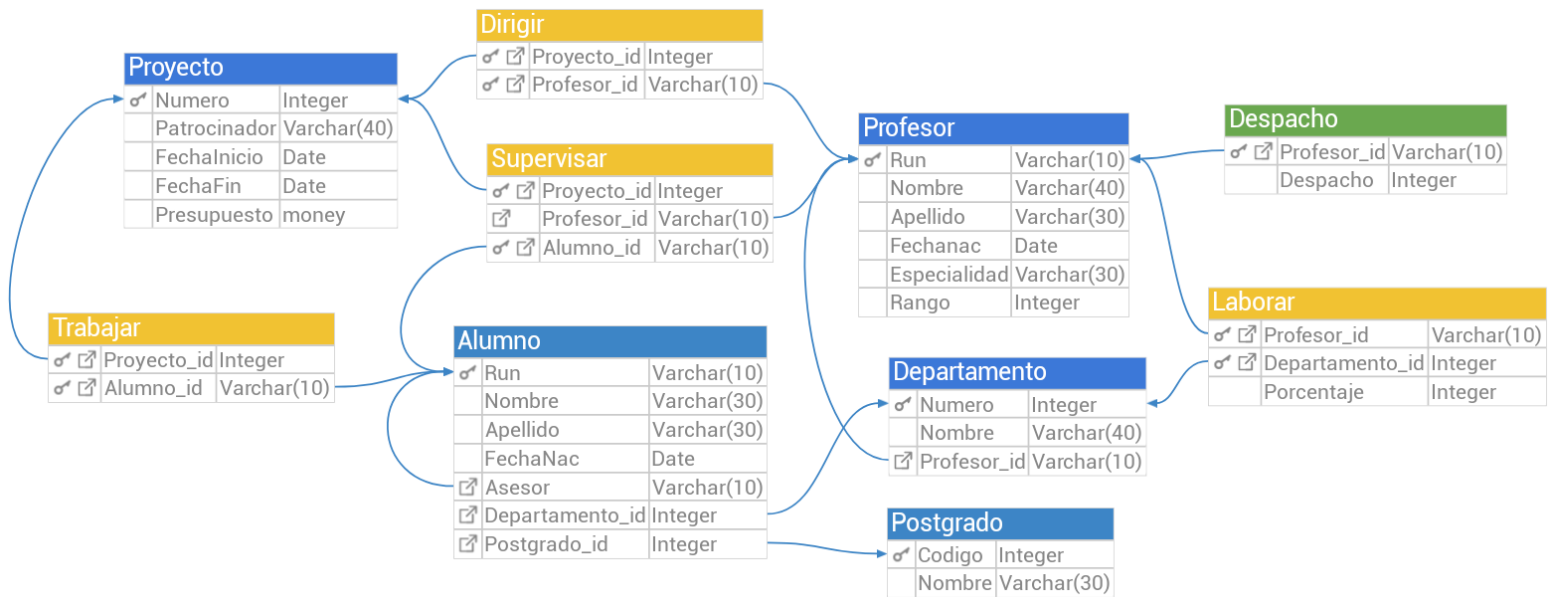


Figura 2.22: Modelo Relacional Normalizado de la universidad

Juaramir: Quiero aclarar que para el resto del informe se asumirá que el despacho es una oficina independiente del profesor que administra el departamento, por lo que no se considerará esta normalización en el resto del informe. **END.**

Capítulo 3

Lenguaje de Definición de Datos

3.1. Creación de Tablas

En el Código 3.1 se presentan las eliminaciones de las tablas en caso de ser necesario, en el Código 3.2 se presentan los `create table` del sistema, basados en el modelo relacional normalizado de la Figura 2.22.

Código 3.1: Eliminación de las Tablas

```
1 DROP TABLE IF EXISTS Proyecto;  
2 DROP TABLE IF EXISTS Departamento;  
3 DROP TABLE IF EXISTS Profesor;  
4 DROP TABLE IF EXISTS Trabajar;  
5 DROP TABLE IF EXISTS Supervisar;  
6 DROP TABLE IF EXISTS Dirigir;  
7 DROP TABLE IF EXISTS Laborar;  
8 DROP TABLE IF EXISTS Alumno;  
9 DROP TABLE IF EXISTS Postgrado;
```

Código 3.2: Creaciones de Tablas

```
1 CREATE TABLE Proyecto (  
2 Numero INTEGER PRIMARY KEY NOT NULL,  
3 Patrocinador VARCHAR(40),  
4 FechaInicio DATE NOT NULL,  
5 FechaFin DATE,  
6 Presupuesto MONEY);  
7  
8 CREATE TABLE Postgrado (  
9 Codigo INTEGER PRIMARY KEY NOT NULL,  
10 Nombre VARCHAR(30) NOT NULL);  
11
```

```
12 CREATE TABLE Profesor (  
13 Run VARCHAR(10) PRIMARY KEY NOT NULL,  
14 Nombre VARCHAR(40),  
15 Apellido VARCHAR(30),  
16 Fechanac DATE,  
17 Especialidad VARCHAR(30),  
18 Rango INTEGER);  
19  
20 CREATE TABLE Departamento (  
21 Numero INTEGER PRIMARY KEY NOT NULL,  
22 Nombre VARCHAR(40),  
23 Despacho INTEGER,  
24 Profesor_id VARCHAR(10) NOT NULL UNIQUE,  
25 FOREIGN KEY (Profesor_id) REFERENCES Profesor(Run) ON DELETE ↵  
    CASCADE ON UPDATE CASCADE);  
26  
27 CREATE TABLE Dirigir (  
28 Proyecto_id INTEGER NOT NULL,  
29 Profesor_id VARCHAR(10) NOT NULL,  
30 FOREIGN KEY (Proyecto_id) REFERENCES Proyecto(Numero) ON DELETE ↵  
    CASCADE ON UPDATE CASCADE,  
31 FOREIGN KEY (Profesor_id) REFERENCES Profesor(Run) ON DELETE ↵  
    CASCADE ON UPDATE CASCADE,  
32 PRIMARY KEY (Proyecto_id, Profesor_id));  
33  
34 CREATE TABLE Laborar (  
35 Profesor_id VARCHAR(10) NOT NULL,  
36 Departamento_id INTEGER NOT NULL,  
37 Porcentaje INTEGER NOT NULL,  
38 FOREIGN KEY (Profesor_id) REFERENCES Profesor(Run) ON DELETE ↵  
    CASCADE ON UPDATE CASCADE,  
39 FOREIGN KEY (Departamento_id) REFERENCES Departamento(Numero) ON ↵  
    DELETE CASCADE ON UPDATE CASCADE  
40 PRIMARY KEY (Profesor_id, Departamento_id));  
41  
42 CREATE TABLE Alumno (  
43 Run VARCHAR(10) PRIMARY KEY NOT NULL,  
44 Nombre VARCHAR(30) NOT NULL,  
45 Apellido VARCHAR(30) NOT NULL,  
46 FechaNac DATE NOT NULL,  
47 Asesor VARCHAR(10) NOT NULL,  
48 Departamento_id INTEGER NOT NULL,
```

```

49 Postgrado_id INTEGER,
50 FOREIGN KEY (Asesor) REFERENCES Alumno(Run) ON DELETE CASCADE ON ↵
    UPDATE CASCADE,
51 FOREIGN KEY (Departamento_id) REFERENCES Departamento(Numero) ON ↵
    DELETE CASCADE ON UPDATE CASCADE,
52 FOREIGN KEY (Postgrado_id) REFERENCES Postgrado(Codigo) ON DELETE ↵
    SET NULL ON UPDATE CASCADE);
53
54 CREATE TABLE Supervisar (
55 Proyecto_id INTEGER NOT NULL,
56 Profesor_id VARCHAR(10) NOT NULL,
57 Alumno_id VARCHAR(10) NOT NULL,
58 FOREIGN KEY (Proyecto_id) REFERENCES Proyecto(Numero) ON DELETE ↵
    CASCADE ON UPDATE CASCADE,
59 FOREIGN KEY (Profesor_id) REFERENCES Profesor(Run) ON DELETE ↵
    CASCADE ON UPDATE CASCADE,
60 FOREIGN KEY (Alumno_id) REFERENCES Alumno(Run) ON DELETE CASCADE ↵
    ON UPDATE CASCADE,
61 PRIMARY KEY (Proyecto_id,Alumno_id));
62
63 CREATE TABLE Trabajar (
64 Proyecto_id INTEGER NOT NULL,
65 Alumno_id VARCHAR(10) NOT NULL,
66 FOREIGN KEY (Proyecto_id) REFERENCES Proyecto(Numero) ON DELETE ↵
    CASCADE ON UPDATE CASCADE,
67 FOREIGN KEY (Alumno_id) REFERENCES Alumno(Run) ON DELETE CASCADE ↵
    ON UPDATE CASCADE,
68 PRIMARY KEY (Proyecto_id,Alumno_id));

```

3.2. Inserción de datos y demostración

A continuación se presentan las sentencias SQL INSERT para poblar las tablas de la base de datos creada con el Código 3.2, solo se presentarán las 10 primeras tuplas, tanto para los insert como para mostrar la tabla cargada en la base de datos.

Código 3.3: Datos para la tabla Profesor

```

1 -- Rango: 1=Instructor , 2=Asistente , 3=Asociado , 4=Titular
2 INSERT INTO Profesor (Run, Nombre, Apellido, FechaNac, ↵
    Especialidad, Rango) VALUES
3 ('12345678-5', 'Ana', 'Pérez', '1975-03-12', 'Bases de Datos', 4) ↵
    ,

```

```

4 ('11222333-K', 'Luis', 'Gómez', '1980-11-25', 'IA', 3),
5 ('98765432-1', 'Carla', 'Morales', '1985-01-18', 'Ing. de Software', 2),
6 ('87654321-0', 'Juan', 'Soto', '1972-07-30', 'Sist. Operativos', 4),
7 ('76543210-K', 'Sofía', 'Castro', '1990-09-01', 'Redes y Seguridad', 2),
8 ('15888777-3', 'Miguel', 'Rojas', '1968-04-15', 'Compiladores', 4),
9 ('13456789-0', 'Elena', 'Vargas', '1982-06-20', 'Computación Gráfica', 3),
10 ('14258369-7', 'Javier', 'Reyes', '1988-12-05', 'HCI', 2),
11 ('10987654-3', 'Lucía', 'Méndez', '1979-10-10', 'Algoritmos', 3),
12 ('16123456-7', 'Pedro', 'Fuentes', '1984-02-28', 'Sist. Distribuidos', 2);

```

Juaramir: Aquí poner Imagen de la tabla con datos en el motor

END.

Código 3.4: Datos para la tabla Postgrado

```

1 INSERT INTO Postgrado (Codigo, Nombre) VALUES
2 (100, 'Magíster en Cs. Computación'),
3 (101, 'Magíster en Ing. de Software'),
4 (200, 'Doctorado en Cs. Computación'),
5 (201, 'Doctorado en IA'),
6 (300, 'Magíster en Data Science'),
7 (301, 'Magíster en Ciberseguridad'),
8 (400, 'Postítulo en Redes'),
9 (401, 'Postítulo en Desarrollo Web'),
10 (102, 'Magíster en Bioinformática'),
11 (202, 'Doctorado en Robótica'),
12 (302, 'Magíster en Videojuegos');

```

Juaramir: Aquí poner Imagen de la tabla con datos en el motor

END.

Código 3.5: Datos para la tabla Proyecto

```

1 INSERT INTO Proyecto (Numero, Patrocinador, FechaInicio, FechaFin,
2 , Presupuesto) VALUES
3 (10, 'ANID-FONDECYT', '2023-03-15', '2026-03-14', 120000000),
4 (11, 'CORFO', '2024-01-20', '2025-07-20', 75000000),
5 (12, 'Interno U.', '2024-08-01', '2025-02-01', 30000000),
6 (13, 'Microsoft Research', '2023-09-01', NULL, 250000000),
7 (14, 'Google AI', '2024-06-01', '2026-05-31', 180000000),

```

```

7 (15, 'ANID-FONDEF', '2025-03-01', '2027-02-28', 95000000),
8 (16, 'Innova Chile', '2023-11-10', NULL, 60000000),
9 (17, 'CERN', '2024-05-01', '2026-04-30', 220000000),
10 (18, 'Tesla', '2025-01-01', NULL, 300000000),
11 (19, 'Interno LabComp', '2024-09-01', '2025-03-01', 25000000);

```

Juaramir: Aquí poner Imagen de la tabla con datos en el motor

END.

Código 3.6: Datos para la tabla Departamento

```

1 INSERT INTO Departamento (Numero, Nombre, Despacho, Profesor_id) ←
  VALUES
2 (1, 'Ingeniería de Software', 301, '98765432-1'),
3 (2, 'Ciencias de la Computación', 210, '12345678-5'),
4 (3, 'Sistemas y Computación', 420, '87654321-0'),
5 (4, 'Redes y Ciberseguridad', 530, '76543210-K'),
6 (5, 'Inteligencia Artificial', 215, '11222333-K'),
7 (6, 'Computación Gráfica y Robótica', 110, '13456789-0'),
8 (7, 'Algoritmia y Teoría', 222, '10987654-3'),
9 (8, 'Sistemas Distribuidos', 333, '16123456-7'),
10 (9, 'Interacción Humano-Computador', 123, '14258369-7'),
11 (10, 'Centro de Investigación', 555, '15888777-3');

```

Juaramir: Aquí poner Imagen de la tabla con datos en el motor

END.

Código 3.7: Datos para la tabla Alumno

```

1 -- El Asesor es NOT NULL, por lo que se crea una cadena de asesoría ←
  a
2 INSERT INTO Alumno (Run, Nombre, Apellido, FechaNac, Asesor, ←
  Departamento_id, Postgrado_id) VALUES
3 ('20111222-3', 'Fernanda', 'Rojas', '2001-05-20', '20333444-5', ←
  2, 100),
4 ('20222333-4', 'Martín', 'Soto', '2002-02-15', '20111222-3', 1, ←
  101),
5 ('20333444-5', 'Camila', 'Vargas', '2000-10-10', '20222333-4', 2, ←
  200),
6 ('19888777-6', 'Diego', 'Castro', '1999-12-01', '20333444-5', 4, ←
  301),
7 ('19777666-5', 'Valentina', 'Morales', '1998-03-25', '20111222-3' ←
  , 3, NULL),
8 ('21123456-7', 'Benjamín', 'Herrera', '2003-01-30', '20222333-4', ←
  5, 201),
9 ('21543210-9', 'Isidora', 'Jiménez', '2002-08-14', '19888777-6', ←

```

```

7, NULL),
10 ('19654321-8', 'Tomás', 'Flores', '1999-07-07', '19777666-5', 6, ←
    102),
11 ('20876543-2', 'Antonia', 'Reyes', '2001-04-18', '21123456-7', 8, ←
    300),
12 ('21001002-K', 'Sebastián', 'Silva', '2003-06-22', '19654321-8', ←
    9, 202),
13 ('19555444-3', 'Javiera', 'Lagos', '1998-11-11', '20111222-3', 2, ←
    NULL),
14 ('19444333-2', 'Matías', 'Nuñez', '1999-09-09', '20333444-5', 1, ←
    101);

```

Juaramir: Aquí poner Imagen de la tabla con datos en el motor

END.

Código 3.8: Datos para la tabla Dirigir

```

1 INSERT INTO Dirigir (Proyecto_id, Profesor_id) VALUES
2 (10, '12345678-5'), -- Ana dirige proyecto 10
3 (11, '98765432-1'), -- Carla dirige proyecto 11
4 (12, '98765432-1'), -- Carla también dirige proyecto 12
5 (13, '11222333-K'), -- Luis dirige proyecto 13
6 (14, '11222333-K'), -- Luis también dirige proyecto 14
7 (15, '76543210-K'), -- Sofía dirige proyecto 15
8 (16, '87654321-0'), -- Juan dirige proyecto 16
9 (17, '15888777-3'), -- Miguel dirige proyecto 17
10 (18, '13456789-0'), -- Elena dirige proyecto 18
11 (19, '14258369-7'); -- Javier dirige proyecto 19

```

Juaramir: Aquí poner Imagen de la tabla con datos en el motor

END.

Código 3.9: Datos para la tabla Laborar

```

1 INSERT INTO Laborar (Profesor_id, Departamento_id, Porcentaje) ←
    VALUES
2 ('12345678-5', 2, 100), -- Ana en Cs. de la Computación
3 ('11222333-K', 5, 100), -- Luis en IA
4 ('98765432-1', 1, 100), -- Carla en Ing. de Software
5 ('87654321-0', 3, 100), -- Juan en Sistemas y Computación
6 ('76543210-K', 4, 100), -- Sofía en Redes y Ciberseguridad
7 ('15888777-3', 2, 50), -- Miguel 50% en Cs. de la Comp.
8 ('15888777-3', 7, 50), -- y 50% en Algoritmia
9 ('13456789-0', 6, 100), -- Elena en Comp. Gráfica
10 ('10987654-3', 7, 100), -- Lucía en Algoritmia
11 ('16123456-7', 8, 100), -- Pedro en Sist. Distribuidos

```

```
12 ('14258369-7', 9, 100); -- Javier en HCI
```

Juaramir: Aquí poner Imagen de la tabla con datos en el motor

END.

Código 3.10: Datos para la tabla Trabajar

```
1 INSERT INTO Trabajar (Proyecto_id, Alumno_id) VALUES
2 (10, '20111222-3'), -- Fernanda en proyecto 10
3 (10, '20333444-5'), -- Camila en proyecto 10
4 (11, '20222333-4'), -- Martín en proyecto 11
5 (13, '21123456-7'), -- Benjamín en proyecto 13
6 (13, '20111222-3'), -- Fernanda también en proyecto 13
7 (14, '19888777-6'), -- Diego en proyecto 14
8 (15, '21543210-9'), -- Isidora en proyecto 15
9 (17, '19654321-8'), -- Tomás en proyecto 17
10 (18, '20876543-2'), -- Antonia en proyecto 18
11 (18, '21001002-K'), -- Sebastián en proyecto 18
12 (19, '19555444-3'); -- Javiera en proyecto 19
```

Juaramir: Aquí poner Imagen de la tabla con datos en el motor

END.

Código 3.11: Datos para la tabla Supervisar

```
1 -- PK es (Proyecto_id, Alumno_id), lo que implica un único ←
   supervisor por alumno en un proyecto
2 INSERT INTO Supervisar (Proyecto_id, Profesor_id, Alumno_id) ←
   VALUES
3 (10, '12345678-5', '20111222-3'), -- En P10, Ana supervisa a ←
   Fernanda
4 (10, '15888777-3', '20333444-5'), -- En P10, Miguel supervisa a ←
   Camila
5 (11, '98765432-1', '20222333-4'), -- En P11, Carla supervisa a ←
   Martín
6 (13, '11222333-K', '21123456-7'), -- En P13, Luis supervisa a ←
   Benjamín
7 (13, '11222333-K', '20111222-3'), -- En P13, Luis también ←
   supervisa a Fernanda
8 (14, '11222333-K', '19888777-6'), -- En P14, Luis supervisa a ←
   Diego
9 (15, '76543210-K', '21543210-9'), -- En P15, Sofía supervisa a ←
   Isidora
10 (17, '15888777-3', '19654321-8'), -- En P17, Miguel supervisa a ←
   Tomás
11 (18, '13456789-0', '20876543-2'), -- En P18, Elena supervisa a ←
```

```

12      Antonia
(18, '13456789-0', '21001002-K'), -- En P18, Elena también ↵
      supervisa a Sebastián
13 (19, '14258369-7', '19555444-3'); -- En P19, Javier supervisa a ↵
      Javiera

```

Código 3.12: Datos insertados en la tabla Supervisar

```

1 select * from supervisar limit 10;
2 proyecto_id | profesor_id | alumno_id
3 -----+-----+-----
4           10 | 12345678-5 | 20111222-3
5           10 | 15888777-3 | 20333444-5
6           11 | 98765432-1 | 20222333-4
7           13 | 11222333-K | 21123456-7
8           13 | 11222333-K | 20111222-3
9           14 | 11222333-K | 19888777-6
10          15 | 76543210-K | 21543210-9
11          17 | 15888777-3 | 19654321-8
12          18 | 13456789-0 | 20876543-2
13          18 | 13456789-0 | 21001002-K

```

Capítulo 4

Lenguaje de Manipulación de Datos

4.1. Restricciones

Fecha de Inicio del Proyecto La fecha de inicio del proyecto debe ser igual o posterior a la fecha y hora actual del servidor.

Código 4.1: Restricción Fecha Inicio Proyecto

```
1 ALTER TABLE Proyecto ADD CONSTRAINT ↵
    chk_proyecto_fechainicio_valida CHECK (FechaInicio >= ↵
    CURRENT_TIMESTAMP);
2 -- Nota: now() es una función tradicional de PostgreSQL, ↵
    CURRENT_TIMESTAMP es el estándar SQL.
```

Código 4.2: Insert Impedido por la restricción chk_proyecto_fechainicio_valida

```
1 INSERT INTO Proyecto (Numero, Patrocinador, FechaInicio, FechaFin↵
    , Presupuesto) VALUES
2 (10, 'ANID-FONDECYT', '2023-03-15', '2026-03-14', 120000000);
3 ERROR: new row for relation "proyecto" violates check constraint↵
    "chk_proyecto_fechainicio_valida"
4 DETAIL: Failing row contains (10, ANID-FONDECYT, 2023-03-15, ↵
    2026-03-14, $ 120.000.000,00).
```

Dado que se desean almacenar proyectos históricos, se eliminará la restricción chk_proyecto_fechainicio_valida, esto se realiza como se muestra en el Código 4.3.

Código 4.3: Eliminar Restricción Fecha Inicio Proyecto

```
1 ALTER TABLE Proyecto DROP CONSTRAINT ↵
    chk_proyecto_fechainicio_valida;
```

Presupuesto No Negativo El presupuesto asignado a un proyecto no puede ser un valor negativo.

Código 4.4: Restricción Presupuesto no negativo

```

1 ALTER TABLE Proyecto
2 ADD CONSTRAINT chk_proyecto_presupuesto CHECK (Presupuesto >= '0'
  :: MONEY);

```

Coherencia de Fechas del Proyecto Si se especifica una fecha de finalización para un proyecto, esta debe ser posterior a su fecha de inicio.

Código 4.5: Coherencia de Fechas del Proyecto

```

1 ALTER TABLE Proyecto
2 ADD CONSTRAINT chk_proyecto_fechas CHECK (FechaFin IS NULL OR
  FechaFin > FechaInicio);

```

Fecha de Nacimiento Válida La fecha de nacimiento de un profesor debe ser anterior a la fecha actual.

Código 4.6: Fecha de Nacimiento Válida

```

1 ALTER TABLE Profesor
2 ADD CONSTRAINT chk_profesor_fechanac CHECK (FechaNac <
  CURRENT_DATE);

```

Rango del Profesor El rango académico de un profesor debe pertenecer a una lista predefinida de valores válidos (ej. 1:'Titular', 2:'Asociado', 3:'Asistente', 4:'Instructor').

Código 4.7: Rango del Profesor

```

1 ALTER TABLE Profesor
2 ADD CONSTRAINT chk_profesor_rango CHECK (Rango BETWEEN 1 AND 4);

```

Auto-Asesoría Inválida Un alumno no puede ser registrado como su propio asesor.

Código 4.8: Auto-Asesoría Inválida

```

1 ALTER TABLE Alumno
2 ADD CONSTRAINT chk_alumno_no_autoasesor CHECK (Run != Asesor);

```

Porcentaje de Dedicación Válido El porcentaje de tiempo que un profesor labora en un departamento debe ser un valor positivo y no exceder el 100 %.

Código 4.9: Porcentaje de Dedicación Válido

```

1 ALTER TABLE Laborar
2 ADD CONSTRAINT chk_laborar_porcentaje CHECK (Porcentaje > 0 AND
  Porcentaje <= 100);

```

Número de departamento positivo

Código 4.10: Número de departamento positivo

```

1 ALTER TABLE Departamento
2 ADD CONSTRAINT chk_departamento_numero CHECK (Numero > 0);

```

Número de despacho positivo

Código 4.11: Número de despacho positivo

```

1 ALTER TABLE Departamento
2 ADD CONSTRAINT chk_departamento_despacho CHECK (Despacho > 0);

```

Código positivo

Código 4.12: Código positivo

```

1 ALTER TABLE Postgrado
2 ADD CONSTRAINT chk_postgrado_codigo CHECK (Codigo > 0);

```

4.2. Consultas y Vistas

Profesores de máximo rango Selecciona el nombre y la especialidad de todos los profesores que tienen el rango más alto (4).

Código 4.13: Profesores de máximo rango

```

1 SELECT Nombre, Apellido, Especialidad
2 FROM Profesor
3 WHERE Rango = 4;

```

Código 4.14: Resultado Código 4.13

nombre	apellido	especialidad
Ana	Pérez	Bases de Datos
Juan	Soto	Sist. Operativos
Miguel	Rojas	Compiladores

Proyectos sin fecha de fin Obtiene el número, patrocinador y presupuesto de los proyectos que aún no tienen una fecha de finalización definida.

Código 4.15: Proyectos sin fecha de fin

```

1 SELECT Numero, Patrocinador, Presupuesto
2 FROM Proyecto

```

```
3 WHERE FechaFin IS NULL;
```

Código 4.16: Resultado Código 4.15

numero	patrocinador	presupuesto
13	Microsoft Research	\$ 250.000.000,00
16	Innova Chile	\$ 60.000.000,00
18	Tesla	\$ 300.000.000,00

Directores de Departamento (Join Implícito) Muestra el nombre de cada departamento y el nombre del profesor que lo dirige.

Código 4.17: Directores de Departamento (Join Implícito)

```
1 SELECT
2   D.Nombre AS NombreDepartamento,
3   P.Nombre AS NombreDirector,
4   P.Apellido AS ApellidoDirector
5 FROM Departamento AS D, Profesor AS P
6 WHERE D.Profesor_id = P.Run;
```

Proyectos y sus Directores (INNER JOIN) Lista todos los proyectos junto con el nombre completo del profesor que los dirige.

Código 4.18: Proyectos y sus Directores (INNER JOIN)

```
1 SELECT
2   Proy.Numero AS ProyectoID,
3   Proy.Patrocinador,
4   Prof.Nombre AS Director,
5   Prof.Apellido AS ApellidoDirector
6 FROM Proyecto AS Proy
7 INNER JOIN Dirigir AS D
8   ON Proy.Numero = D.Proyecto_id
9 INNER JOIN Profesor AS Prof
10  ON D.Profesor_id = Prof.Run;
```

Presupuesto total por patrocinador Calcula la suma total de los presupuestos de los proyectos, agrupados por cada patrocinador.

Código 4.19: Presupuesto total por patrocinador

```
1 SELECT Patrocinador,
2   SUM(Presupuesto) AS PresupuestoTotal
3 FROM Proyecto
```

```

4 GROUP BY Patrocinador
5 ORDER BY PresupuestoTotal DESC;

```

Profesores que dirigen más de un proyecto Utiliza COUNT para contar los proyectos por profesor y HAVING para filtrar y mostrar solo a aquellos que dirigen más de uno.

Código 4.20: Profesores que dirigen más de un proyecto

```

1 SELECT P.Nombre, P.Apellido,
2     COUNT(D.Proyecto_id) AS ProyectosDirigidos
3 FROM Profesor AS P
4 INNER JOIN Dirigir AS D
5     ON P.Run = D.Profesor_id
6 GROUP BY P.Run, P.Nombre, P.Apellido
7 HAVING COUNT(D.Proyecto_id) > 1;

```

Alumnos en programas de Magíster (Uso de IN) Encuentra a todos los alumnos cuyos programas de postgrado contienen la palabra 'Magíster'.

Código 4.21: Alumnos en programas de Magíster (Uso de IN)

```

1 SELECT Nombre, Apellido
2 FROM Alumno
3 WHERE
4     Postgrado_id IN (
5         SELECT Codigo
6         FROM Postgrado
7         WHERE Nombre LIKE 'Magíster%'
8     );

```

Departamentos con profesores que trabajan al 100 % (Uso de EXISTS) Lista los departamentos donde existe al menos un profesor que dedica el 100 % de su tiempo a ese departamento.

Código 4.22: Departamentos con profesores que trabajan al 100 % (Uso de EXISTS)

```

1 SELECT Nombre
2 FROM Departamento AS D
3 WHERE EXISTS (
4     SELECT 1
5     FROM Laborar AS L
6     WHERE L.Departamento_id = D.Numero AND L.Porcentaje = 100
7 );

```

Alumnos sin proyecto (Usando LEFT JOIN) La forma más común y eficiente. Se traen todos los alumnos y se unen con la tabla Trabajar. Aquellos que no tienen correspondencia (t.Proyecto_id es NULL) son los que no trabajan en proyectos.

Código 4.23: Alumnos sin proyecto (Usando LEFT JOIN)

```

1 SELECT A.Run, A.Nombre, A.Apellido
2 FROM Alumno AS A
3 LEFT JOIN Trabajar AS T
4   ON A.Run = T.Alumno_id
5 WHERE T.Proyecto_id IS NULL;

```

Alumnos sin proyecto (Usando NOT EXISTS) Selecciona a los alumnos para los cuales no existe una entrada en la tabla Trabajar que los vincule a un proyecto.

Código 4.24: Alumnos sin proyecto (Usando NOT EXISTS)

```

1 SELECT A.Nombre, A.Apellido
2 FROM Alumno AS A
3 WHERE NOT EXISTS (
4     SELECT 1
5     FROM Trabajar AS T
6     WHERE T.Alumno_id = A.Run
7 );

```

Creación de una vista ProyectoDetallado Crea una vista que simplifica las consultas futuras al unir la información clave de los proyectos: el proyecto en sí, el director y el departamento del director.

Código 4.25: Creación de una vista ProyectoDetallado

```

1 CREATE VIEW VistaProyectoDetallado AS
2 SELECT
3     Proy.Numero AS ProyectoNumero,
4     Proy.Patrocinador,
5     Proy.Presupuesto,
6     Prof.Nombre AS NombreDirector,
7     Prof.Apellido AS ApellidoDirector,
8     Dep.Nombre AS DepartamentoDirector
9 FROM Proyecto AS Proy
10 INNER JOIN Dirigir AS Dir
11   ON Proy.Numero = Dir.Proyecto_id
12 INNER JOIN Profesor AS Prof
13   ON Dir.Profesor_id = Prof.Run
14 INNER JOIN Departamento AS Dep

```

```
15 | ON Prof.Run = Dep.Profesor_id;
```

Uso de la vista para filtrar proyectos Una vez creada la vista, se puede consultar como si fuera una tabla normal para encontrar, por ejemplo, todos los proyectos dirigidos por un profesor del departamento de 'Inteligencia Artificial'.

Código 4.26: Uso de la vista para filtrar proyectos

```
1 | SELECT *
2 | FROM VistaProyectoDetallado
3 | WHERE DepartamentoDirector = 'Inteligencia Artificial';
```

Supervisión de Alumnos en Proyectos Muestra una lista detallada de qué alumno es supervisado por qué profesor y en qué proyecto, mostrando los nombres en lugar de los IDs.

Código 4.27: Supervisión de Alumnos en Proyectos

```
1 | SELECT
2 |   P.Nombre AS NombreProyecto,
3 |   Al.Nombre AS NombreAlumno,
4 |   Al.Apellido AS ApellidoAlumno,
5 |   Prof.Nombre AS NombreSupervisor,
6 |   Prof.Apellido AS ApellidoSupervisor
7 | FROM Supervisar AS S
8 | INNER JOIN Proyecto AS P
9 |   ON S.Proyecto_id = P.Numero
10 | INNER JOIN Alumno AS Al
11 |   ON S.Alumno_id = Al.Run
12 | INNER JOIN Profesor AS Prof
13 |   ON S.Profesor_id = Prof.Run
14 | ORDER BY NombreProyecto;
```

Postgrado con más alumnos Encuentra el programa de postgrado con el mayor número de alumnos inscritos usando una combinación de GROUP BY, ORDER BY, LIMIT y una subconsulta.

Código 4.28: Postgrado con más alumnos

```
1 | SELECT PG.Nombre,
2 |   COUNT(A.Run) AS NumeroDeAlumnos
3 | FROM Postgrado AS PG
4 | INNER JOIN Alumno AS A
5 |   ON PG.Codigo = A.Postgrado_id
6 | GROUP BY PG.Nombre
7 | ORDER BY NumeroDeAlumnos DESC
```

8 | `LIMIT 1;`